

浙江大学



多机械臂搬运调度与模式选择

运筹学课程大作业

团队成员：修嘉卿 3240104703

傅安澜 3240104374

王沐灵 3240101575

专业：自动化（电气学院）

指导老师：何衍

日期：2026 年 6 月 16 日

目录

摘要	3
1 前言	3
1.1 研究背景与问题定位	3
1.2 问题特点与难点	4
1.3 项目思路	4
2 问题描述	5
2.1 问题场景	5
2.2 集合、参数与执行模式	6
2.2.1 任务处理时间	7
2.2.2 任务处理能耗	7
2.3 决策变量	8
2.4 约束条件	9
2.5 目标规划模型	10
3 主问题：整数规划与目标规划	11
3.1 CP-SAT 求解器	12
3.1.1 求解对象	12
3.1.2 CP-SAT 的搜索机制	12
3.2 融合剪枝的隐枚举法	13
3.2.1 决策过程	13
3.2.2 剪枝决策	17
3.3 启发式算法	17
3.3.1 初始解构造	17
3.3.2 大邻域改进	18
3.4 热启动后的隐枚举法	20
3.5 二臂/三臂模式选择在主问题中的实现	21
3.6 与非线性扩展的衔接	22
4 延伸问题：非线性规划	23
4.1 问题一：速度-能耗非线性优化问题	24
4.1.1 问题分析	24
4.1.2 实现过程	26
4.1.3 KKT 验证条件	28
4.2 问题二：鲁棒速度模式选择问题	29
4.2.1 问题分析	29
4.2.2 实现过程	31
4.2.3 KKT 验证条件	34

5	数据分析与结果讨论	35
5.1	具体数据	35
5.1.1	主问题数据	35
5.1.2	延伸问题数据	37
5.2	解的评估与结果解释	40
5.2.1	主问题解的评估	41
5.2.2	非线性延伸问题解的评估	41
5.3	方法对比	43
5.3.1	主问题四种方法对比	43
5.3.2	延伸问题方法对比	44
6	结论	45
6.1	研究结论	45
6.2	研究亮点	46
6.3	心得体会	47
7	思考	48
7.1	主问题是否可以理解为一部分动态规划?	48
7.2	匈牙利算法为何不能使用?	48
8	附件清单	49
9	参考文献	49
10	分工	50

摘要

本项目面向多机械臂搬运场景中任务分配、执行排序与系统配置选择相互耦合的问题，本文以圆形作业区域内多类别物块搬运为研究对象，构建了一个兼具单臂搬运、双臂协同搬运、目标收集盒约束和机械臂资源互斥约束的运筹学优化模型。

在主问题中，将物块搬运任务抽象为一组可选执行模式，并建立包含模式选择变量、作业区间变量、机械臂衔接变量和资源互斥约束的整数规划与目标规划模型。针对该组合优化问题，本文设计并实现了四类求解方法：以 CP-SAT 的标准求解器方法作为对照、融合剪枝的隐枚举法、启发式算法，以及热启动后的隐枚举法。在实验结果中，CP-SAT 求解器、融合剪枝的隐枚举法和热启动后的隐枚举法在最优解下的平均完工时间与平均能耗上保持一致，说明搜索方法能够达到标准求解器结果；启发式算法计算时间更短，适合快速生成较优可行解。

在模式选择分析与延伸问题中，首先进一步比较二臂与三臂系统配置的差异，三臂模式相对于二臂模式平均可节省约 31.24% 的完成时间，但平均能耗增加约 4.46%。这说明三臂系统能够显著增强并行搬运能力，但其优势并非无条件成立，而取决于决策者对时间收益和能耗代价的权衡。为进一步刻画运行速度对系统表现的影响，本文在主问题结果基础上引入速度倍率变量，建立速度、时间和能耗之间的非线性模型，并采用内点制约函数法与 0.618 一维搜索进行求解，同时利用 KKT 条件检查数值解的一阶最优性。随后又建立鲁棒速度模式选择模型，要求同一任务规模下所有随机 seed 共用统一速度，并最小化最坏情形能耗，以分析任务空间分布波动下二臂与三臂模式的稳健性差异。

总而言之，本文整体从 **0-1 整数规划与目标规划的主问题与非线性规划的延伸问题** 展开分析，体现了整数规划、目标规划、隐枚举法、非线性规划、内点法、SLP 法、KKT 条件等运筹学方法的综合应用，也揭示了多机械臂系统中完成时间、能耗和稳健性之间的权衡关系。研究结果说明，多机械臂搬运系统并不是机械臂数量越多越优，也不是运行速度越快越优，而应根据任务结构、协同需求、工期要求和能耗约束进行综合决策。

关键词：多机械臂调度；整数规划；目标规划；隐枚举法；热启动；非线性规划；KKT 条件

1 前言

1.1 研究背景与问题定位

在智能制造、仓储分拣、柔性装配和自动化物流等场景中，机械臂已经不再只是单独执行固定动作的末端设备，而越来越多地作为系统级作业资源参与多任务协同。与单机械臂系统相比，多机械臂系统能够在同一作业区域内并行完成多个搬运、抓取或分拣任务，从而缩短整体作业时间，提高设备利用率。然而，机械臂数量增加之后，系统并不会自动变得更优。不同机械臂之间可能竞争同一空间区域，多个任务可能同时请求同一机械臂，部分大尺寸或高重量物块还需要两只机械臂协同搬运。因此，多机械臂系统的关键问题之一并不是能不能运动，而是如何安排任务。

本文不讨论机械臂底层关节控制、轨迹跟踪或动力学建模，而是从任务层调度角度研究：在给定物块类型、空间位置、可行执行模式、作业时间和能耗参数的条件下，如

何合理安排任务执行机械臂、搬运顺序与系统运行模式，从而在保证可行性的基础上兼顾完成时间、能耗和资源利用效果。具体而言，本文将一个圆形作业区域内的多类别物块搬运过程作为研究对象。不同类别物块具有不同的搬运要求：部分任务可以由单只机械臂独立完成，部分任务需要两只机械臂同步协同完成；不同类型物块对应不同目标收集盒；不同机械臂具有各自的初始位置、可达范围和任务衔接代价。由此形成的问题不是简单的平均分配任务，而是一个同时包含任务分配、执行排序、协同模式选择、资源互斥和多目标评价的组合优化问题。

1.2 问题特点与难点

本文研究的多机械臂搬运调度问题具有以下几个特点：

1. 任务执行模式具有离散选择性。对于单臂任务，需要决定由哪一只机械臂执行；对于双臂协同任务，需要决定由哪两只机械臂共同执行，二臂系统和三臂系统下的可行模式集合不同，尤其在三臂场景中，双臂协同任务可以选择的机械臂组合更多，虽然增强了并行能力，但也显著扩大了搜索空间。
2. 任务之间存在明显的资源互斥关系。同一机械臂不能同时执行多个任务，两只机械臂协同执行任务时还需要同步开始和结束。若不处理这些互斥关系，即使每个任务本身都可执行，整体调度方案也可能在时间线上发生冲突。因此，模型不仅要决定任务由谁执行，还必须决定任务之间的先后衔接关系。
3. 系统评价目标具有多层次性。最大完成时间反映整体作业效率，是调度问题中最重要的指标；能耗反映系统运行代价；负载均衡则反映多机械臂之间的任务分配是否合理。实际应用中，这些目标并不总是完全一致。例如，增加第三只机械臂通常可以缩短完成时间，但也可能带来额外能耗；提高运行速度可以压缩工期，但可能使能耗上升。
4. 离散调度与连续运行参数之间存在耦合关系。前面的任务分配和排序属于整数规划和组合优化问题，而机械臂速度倍率的选择则属于连续非线性优化问题。若把二者完全合并为一个统一模型，求解难度会迅速增大。本文采用“先离散调度、后连续调速”的分层思路：先求得二臂和三臂场景下的调度方案，再在已有调度结果基础上优化速度倍率，并进一步进行鲁棒性分析。

1.3 项目思路

运筹学中的整数规划、目标规划、分支定界、隐枚举法和启发式算法为该类问题提供了重要工具。整数规划能够较完整地表达任务选择、资源占用、时间先后和逻辑互斥关系，适合构建标准数学模型；CP-SAT 求解器具有较强的通用性和稳定性，可作为标准求解器对照组；隐枚举法和分支定界思想则通过逐步扩展候选解、维护当前最优上界和构造下界进行剪枝，适合处理具有明显离散结构的组合调度问题；启发式算法虽然通常不直接提供全局最优性证明，但能够快速得到质量较好的可行解，在实际工程问题中具有较强实用价值。

基于上述方法，本文形成了四条主问题求解路径。第一条路径使用 CP-SAT 求解器求解整数规划与目标规划模型，用于提供基准结果。第二条路径为融合剪枝的隐枚举法，即在隐枚举搜索过程中引入完工时间下界、能耗下界、机械臂负载下界和中心安全区互斥判断，以减少无效搜索。第三条路径为启发式算法，主要用于根据任务结构快速生成较优可行解。第四条路径为热启动后的隐枚举法，即先由启发式算法提供初始可行解和上界，再由隐枚举法继续搜索与剪枝，从而兼顾启发式速度和精确搜索质量。

在主调度模型之外，本文进一步引入速度-能耗非线性优化。该部分并不重新改变任务分配和执行顺序，而是在已有调度结果上选择单臂任务速度倍率和双臂协同任务速度倍率。这样既避免了离散变量与连续变量完全耦合带来的求解困难，又能够分析速度调整对完成时间和能耗的影响。为了进一步考虑随机任务位置带来的波动，本文还建立鲁棒速度模式选择模型，要求同一任务规模下不同 seed 共用同一组速度，并比较二臂和三臂模式在最坏情形能耗下的表现。

2 问题描述

本项目关注的是多机械臂搬运任务的运筹学调度求解与二臂/三臂运行模式选择，不是机器人运动学、动力学建模或轨迹控制问题；机械臂在本文中主要作为可分配的作业资源参与调度优化。

因此，本文并不研究机械臂关节角、末端轨迹、动力学方程或控制器设计，而是将多机械臂搬运过程抽象为一个具有任务分配、执行排序、资源互斥、安全区占用和多目标优化特征的调度问题。本章主要回答三个问题：第一，实际搬运任务如何抽象为任务集合与执行模式；第二，时间、能耗和安全约束如何用数学符号表达；第三，如何建立同时考虑最大完工时间、总能耗和负载均衡的优化模型。

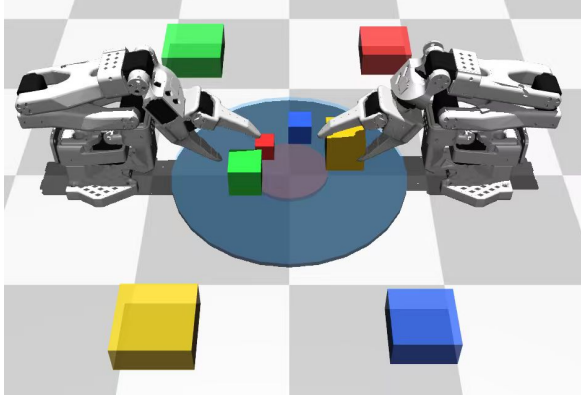
2.1 问题场景

考虑一个圆形作业区域，区域中随机放置若干待搬运方块。每个方块具有类型、重量、初始位置和目标收集盒。系统中可配置两只或三只机械臂，机械臂从各自的待机位置出发，将方块搬运到对应类型的收集盒中。四类方块的任务属性如下：

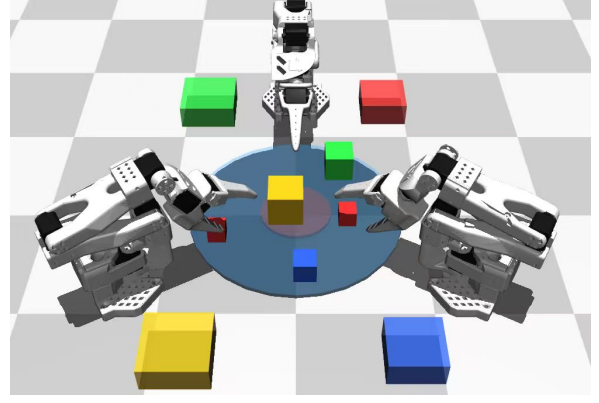
表 1 方块类型与任务属性

方块类型	任务类型	所需机械臂数	目标收集盒	相对搬运难度
Type1	单臂任务	1	Box1	小而轻
Type2	单臂任务	1	Box2	小但较重
Type3	双臂协同任务	2	Box3	大且需协同
Type4	双臂协同任务	2	Box4	大且较重，协同代价高

图 1 给出同一组任务实例在两臂系统和三臂系统下的工作空间示意。图中蓝色圆形区域表示方块随机生成的作业区域，橙色虚线圆表示中心安全参考区，方形标记表示机械臂基座，菱形标记表示四类收集盒，圆点表示待搬运方块，虚线表示方块到对应收集盒的搬运方向。方块旁标注的机械臂组合为该案例求解得到的任务分配结果。



(a) 两臂系统示意



(b) 三臂系统示意

图 1 多机械臂分类搬运任务的工作空间示意

在两臂系统中，双臂协同任务只能由两只机械臂共同完成；在三臂系统中，双臂协同任务可以从三只机械臂中选择任意两只共同完成。因此，三臂系统一方面增加了可选执行模式和并行能力，可能缩短最大完工时间；另一方面也会带来额外的系统固定能耗和协同协调代价。本文后续案例分析正是围绕这一时间-能耗权衡展开。

此外，作业区域中心附近设置安全区。当某个任务的搬运路径经过中心安全区时，该任务组需要占用中心区资源。这里的中心区不是单只机械臂层面的互斥资源，而是任务组层面的共享安全资源：同一个双臂协同任务内部的两只机械臂允许同时进入中心区，但不同任务组不能同时占用中心区。

2.2 集合、参数与执行模式

设任务集合为

$$\mathcal{J} = \{1, 2, \dots, n\}.$$

两臂系统和三臂系统的机械臂集合分别为

$$\mathcal{A}^{(2)} = \{\text{arm1}, \text{arm2}\}, \quad \mathcal{A}^{(3)} = \{\text{arm1}, \text{arm2}, \text{arm3}\}.$$

下文若不特别区分两臂或三臂，统一记机械臂集合为 $\mathcal{A}$ ，其规模为 $|\mathcal{A}| = m$ ，其中 $m \in \{2, 3\}$ 。

每个任务 $j \in \mathcal{J}$ 对应一个方块。为便于后续建模，表 2 汇总主要任务参数。

表 2 任务与系统参数说明

符号	含义
p_j	任务 j 对应方块的初始位置
q_j	任务 j 对应目标收集盒的位置
b_j	方块类型, $b_j \in \{1, 2, 3, 4\}$
r_j	完成任务 j 所需机械臂数量
w_j	方块重量
d_j	方块从初始位置到目标收集盒的搬运距离

不同方块类型决定任务所需机械臂数量：

$$r_j = \begin{cases} 1, & b_j \in \{1, 2\}, \\ 2, & b_j \in \{3, 4\}. \end{cases}$$

搬运距离定义为

$$d_j = \|p_j - q_j\|_2.$$

对每个任务 j ，根据可达性和所需机械臂数构造可行执行模式集合 $\mathcal{M}_j$ 。一个执行模式 $k \in \mathcal{M}_j$ 包含以下信息：

$$k = (j, A_k, t_k, e_k, z_k),$$

其含义如表 3 所示。

表 3 执行模式 k 的组成

符号	含义
j	该模式对应的任务编号
A_k	执行该模式占用的机械臂集合，满足 $A_k \subseteq \mathcal{A}$ 且 $ A_k = r_j$
t_k	该模式下的任务处理时间
e_k	该模式下的任务处理能耗
z_k	是否经过中心安全区， $z_k = 1$ 表示经过， $z_k = 0$ 表示不经过

2.2.1 任务处理时间

本文将机械臂完成一个任务的时间分为两部分：一是执行当前任务本身所需的处理时间，二是任务之间的空载转移时间。这里先定义任务处理时间；空载转移时间将在后续序列相关约束中单独建模。

任务处理时间只描述机械臂到达方块位置后，将方块搬运到目标收集盒所需的时间。它由负载搬运时间、抓取/放置服务时间和安全缓冲时间组成。设单臂负载速度为 v_s ，双臂协同负载速度为 v_d ，则

$$t_k = \begin{cases} (d_j/v_s + T_s + T_{\text{safe}}) \cdot \kappa_t, & |A_k| = 1, \\ (d_j/v_d + T_d + T_{\text{safe}}) \cdot \kappa_t, & |A_k| = 2, \end{cases}$$

其中 d_j/v_s 或 d_j/v_d 表示带负载搬运阶段的行走时间； T_s, T_d 分别表示单臂和双臂任务的抓取、夹持稳定与放置服务时间； T_{safe} 为调度层面的等效安全缓冲时间，用于表示对准、同步、放置稳定等未单独细分的短时间余量。 κ_t 为时间离散化尺度，代码实现中取 $\kappa_t = 100$ ，即将 1 秒连续时间转换为 100 个整数 tick，以便整数规划和分枝定界算法处理。

2.2.2 任务处理能耗

任务处理能耗采用等效能耗模型描述，用于在调度层面区分不同方块类型的搬运难度。该能耗并不是对电机电流、关节力矩等底层物理量的精确预测，而是根据方块重

量、搬运距离、任务持续时间和双臂协同复杂度构造的综合评价指标。这里定义的处理能耗记为 e_k ，即后文系统总能耗公式中 $\sum e_k x_k$ 对应的任务执行能耗项。

对任务 j 的某个执行模式 k ，其处理能耗可表示为

$$e_k = \kappa_e \left(\phi_j E_{jk}^{\text{load}} + E_{jk}^{\text{grasp}} + |A_k| h_j \frac{t_k}{\kappa_t} + \sigma_j \eta_m \right).$$

其中， E_{jk}^{load} 为基础负载搬运能耗，与方块重量和搬运距离有关； E_{jk}^{grasp} 为抓取、夹持建立和放置动作产生的服务能耗； $|A_k| h_j t_k / \kappa_t$ 表示任务执行期间参与机械臂维持夹持与稳定搬运所需的保持能耗； $\sigma_j \eta_m$ 表示双臂协同任务中的额外同步协调代价及系统规模修正。 κ_e 为能耗离散化尺度，用于将连续能耗转换为整数能耗值，以便与整数规划和分支定界算法统一。

为刻画不同方块类型对能耗的影响，引入表 4 中的类型相关参数。

表 4 方块类型相关能耗参数

参数	名称	含义
ϕ_j	载荷修正系数	放大搬运难度对能耗的影响
h_j	保持功率	单位时间内的夹持与稳定搬运代价
σ_j	同步能耗	双臂协同任务的一次性协调代价

对于 Type1 和 Type2 单臂任务，任务不涉及双臂同步，因此令 $\sigma_j = 0$ ，且 $|A_k| = 1$ 。对于 Type3 和 Type4 双臂协同任务， σ_j 取正值， $|A_k| = 2$ ，用于刻画两只机械臂在同步支撑、同步移动和协调放置过程中的额外能耗。两臂系统中 $\eta_m = 1$ ；三臂系统虽然每个双臂任务仍只使用两只机械臂，但可选组合更多、调度协调更复杂，因此 η_m 可取略大于 1 的值。其中 Type4 方块的搬运难度更高，因此其 ϕ_j 、 h_j 或 σ_j 可设置为更大的取值。

2.3 决策变量

为了同时描述任务模式选择、时间安排和机械臂作业顺序，定义如下决策变量。

- $x_k \in \{0, 1\}$: 若任务执行模式 k 被选择，则 $x_k = 1$ ，否则为 0。
- S_k, E_k : 模式 k 对应任务处理段的开始时间和结束时间。
- $y_{a,u,v} \in \{0, 1\}$: 对机械臂 a ，若其作业顺序中模式 u 后紧接模式 v ，则 $y_{a,u,v} = 1$ 。
- $C_{\max}$: 所有任务的最大完工时间。
- E_{tot} : 系统总能耗。
- L_a : 机械臂 a 的总负载时间，包括任务处理时间和空载转移时间。
- B : 机械臂负载不均衡度，即最大负载与最小负载之差。

其中, $y_{a,u,v}$ 用于描述序列相关 **setup**。机械臂执行完一个任务后, 需要从上一任务目标收集盒移动到下一任务的方块初始位置; 若该任务是该机械臂的第一个任务, 则需要从机械臂待机位置移动到该方块位置。这部分空载转移时间和能耗都与任务顺序有关。

2.4 约束条件

1. 任务唯一执行模式约束

每个任务必须且只能选择一种可行执行模式:

$$\sum_{k \in \mathcal{M}_j} x_k = 1, \quad \forall j \in \mathcal{J}.$$

该约束保证每个方块都被搬运一次, 且不会被多个模式重复执行。

2. 时间一致性约束

若模式 k 被选择, 则其结束时间等于开始时间加处理时间:

$$E_k = S_k + t_k, \quad \forall k \text{ with } x_k = 1.$$

在整数规划实现中, 未被选择的模式通过可选区间变量处理; 在自编算法中, 未被选择的模式不会进入调度序列。

3. 机械臂资源互斥约束

同一只机械臂在同一时刻不能执行两个任务。若两个被选择模式 u, v 均占用机械臂 a , 则必须满足二者在时间轴上不重叠:

$$[S_u, E_u) \cap [S_v, E_v) = \emptyset, \quad \forall a \in A_u \cap A_v.$$

基础模型中该约束通过 NoOverlap 资源约束实现; 自编求解器中则通过维护每只机械臂的可用时间递推生成可行调度。

4. 序列相关空载转移约束

设机械臂 a 执行完模式 u 后紧接执行模式 v , 则模式 v 的开始时间必须不早于模式 u 的结束时间加空载转移时间:

$$S_v \geq E_u + \tau_{u,v}, \quad \text{if } y_{a,u,v} = 1.$$

其中

$$\tau_{u,v} = \frac{\|q_u - p_v\|_2}{v_0} \cdot \kappa_t,$$

v_0 为空载移动速度。若 v 是机械臂 a 的第一个任务，则

$$S_v \geq \frac{\|o_a - p_v\|_2}{v_0} \cdot \kappa_t,$$

其中 o_a 为机械臂 a 的待机位置。

相应的空载转移能耗记为

$$\epsilon_{u,v} = \alpha_0 \|q_u - p_v\|_2 \cdot \kappa_e,$$

首任务从待机位置出发时同理计算。该项将计入系统总能耗。

5. 中心安全区互斥约束

设 $\mathcal{C}$ 为经过中心安全区的已选任务模式集合。对于任意两个不同任务组 $u, v \in \mathcal{C}$ ，其中心区占用时间不能重叠：

$$[S_u, E_u) \cap [S_v, E_v) = \emptyset, \quad u \neq v.$$

该约束体现中心安全区在任务组层面的互斥。需要强调的是，对于同一个双臂协同任务，其内部两只机械臂属于同一任务组，不构成彼此冲突。

6. 最大完工时间、总能耗与负载均衡约束

最大完工时间满足

$$C_{\max} \geq E_k, \quad \forall k \text{ with } x_k = 1.$$

系统总能耗由处理能耗、空载转移能耗和系统级固定开销组成：

$$E_{\text{tot}} = \sum_{j \in \mathcal{J}} \sum_{k \in \mathcal{M}_j} e_k x_k + \sum_{a \in \mathcal{A}} \sum_{(u,v)} \epsilon_{u,v} y_{a,u,v} + m E_{\text{sys}},$$

其中 E_{sys} 为单只机械臂的启动、上电、伺服保持和控制通信等效固定能耗。

机械臂负载 L_a 包括该机械臂承担的处理时间和空载转移时间。负载均衡指标定义为

$$B = \max_{a \in \mathcal{A}} L_a - \min_{a \in \mathcal{A}} L_a.$$

2.5 目标规划模型

本问题同时关注完成效率、运行能耗和机械臂利用均衡。由于三个目标的重要性不同，本文不采用简单加权求和，而采用序贯多目标优化思想：先优化最高优先级目标，再在保持高优先级目标最优的前提下优化下一目标。

第一阶段以最大完工时间最小为目标：

$$P_1 : \quad \min C_{\max}.$$

记第一阶段得到的最优值为 $C_{\max}^*$ 。第二阶段在保持最优完工时间不变的条件下，最小化系统总能耗：

$$P_2 : \quad \min E_{\text{tot}}, \quad \text{s.t. } C_{\max} = C_{\max}^*.$$

记第二阶段得到的最优能耗为 E_{tot}^* 。第三阶段在保持前两级目标最优的条件下，最小化机械臂负载不均衡度：

$$P_3 : \quad \min B, \quad \text{s.t. } C_{\max} = C_{\max}^*, E_{\text{tot}} = E_{\text{tot}}^*.$$

该目标结构表示：首先保证整体任务尽快完成，其次在不牺牲时间目标的前提下降低能耗，最后在前两者基础上改善机械臂负载均衡。这样处理避免了人为选择加权系数，也使求解结果的优先级解释更加明确。

综上，本文的核心模型可以理解为一个带有可选执行模式、并行机器资源约束、序列相关 setup、中心安全区互斥和多目标优先级的调度优化模型。后续第 3 章将在该模型基础上，分别设计 CP-SAT 求解器、融合剪枝的隐枚举法、启发式算法和热启动后的隐枚举法四类求解方法，并在第 4 章进一步讨论速度-能耗层的非线性扩展与二臂/三臂模式选择结果。

3 主问题：整数规划与目标规划

本章围绕该主问题展开，重点说明在给定任务实例、执行模式集合、资源约束和目标规划结构后，如何求解任务分配、机械臂组合选择和作业顺序。

主问题共采用四种方法进行求解与对比，分别为 CP-SAT 求解器、融合剪枝的隐枚举法、启发式算法和热启动后的隐枚举法。其中，CP-SAT 求解器作为成熟标准求解器，主要用于提供对照组和基准解；融合剪枝的隐枚举法是在隐枚举思想基础上加入完工时间下界、能耗下界、机械臂负载下界和中心安全区互斥下界的自编精确搜索方法；启发式算法不是课内直接给出的标准算法，而是结合本项目任务结构自主设计的快速构造与局部改进方法；热启动后的隐枚举法则进一步将启发式算法与融合剪枝的隐枚举法结合，用启发式算法提供初始上界，再交给隐枚举搜索继续剪枝优化。

本文的总体求解框架如图 2 所示。输入任务实例后，首先构造每个任务的可行执行方式；随后分别采用 CP-SAT 求解器、融合剪枝的隐枚举法、启发式算法和热启动后的隐枚举法进行调度求解；最后对不同系统配置和速度策略进行分析。



图 2 本文优化方法总体框架

上述四种方法的作用并不完全相同。CP-SAT 求解器相当于标准求解器对照组，用于验证模型结果并提供基准解；融合剪枝的隐枚举法是本文主问题的自编精确搜索方法，在物块数量较少时可以较快完成搜索，但随着物块数量增加，候选任务-执行模式组合迅速增多，求解速度会明显下降；启发式算法用于快速生成较优可行解，它不是课内某一现成算法的直接套用，而是根据本项目的多机械臂任务特点自主设计；热启动后的隐枚举法则将启发式算法与融合剪枝的隐枚举法融合，利用启发式算法先得到较好的初始上界，再加速后续隐枚举过程中的剪枝判断。

3.1 CP-SAT 求解器

CP-SAT 求解器是本文主问题的标准求解器对照组，用其结果作为后续自编求解方法的基准参考。

3.1.1 求解对象

为说明 CP-SAT 接收到的对象， x_k 表示执行模式 k 是否被选择， $[S_k, E_k)$ 表示该模式对应的作业区间， $y_{a,u,v}$ 表示机械臂 a 是否在模式 u 后紧接执行模式 v 。也就是说，CP-SAT 需要同时决定模式选择、作业区间位置和同一机械臂上的前后衔接关系。

CP-SAT 接收到的不是一条已经排好的任务序列，而是一组尚未确定的逻辑变量和整数时间变量。求解器需要同时决定三类内容：每个任务选择哪种执行方式、每个被选方式何时开始和结束、同一机械臂上的任务按什么顺序衔接。

3.1.2 CP-SAT 的搜索机制

CP-SAT 的求解过程可以理解为“约束传播 + 分支搜索 + 冲突学习 + 目标界更新”的循环。

首先，求解器为每个变量维护一个可能取值范围。例如， x_k 的取值范围为 $\{0, 1\}$ ，开始时间 S_k 的取值范围为某个整数时间窗。约束传播会在尚未完全确定解之前不断缩小这些范围。若某个任务只剩一个候选方式没有被排除，唯一选择关系会直接推出该方式必须被选择；若某只机械臂上两个已存在作业的时间窗无法同时容纳，不重叠关系会迫使其中一个作业推迟，或判定当前分支不可行。

当传播无法继续推进时，求解器进入分支搜索。它选择一个尚未确定的逻辑决策或整数变量进行试探，例如假设某个候选方式被选择，或假设两个作业的前后顺序为 u 在 v 之前。每做出一个试探，相关约束又会触发新的传播；若出现矛盾，则说明当前试探组合不能同时成立。

出现冲突后，CP-SAT 不只是简单回退，而会进行冲突分析，提取导致矛盾的一组关键决策，并形成一条避免重复进入类似矛盾的逻辑信息。对于优化问题，求解器还会维护当前最好目标值和可能达到的下界；一旦某个分支已经不可能优于当前最好解，就可以被剪去。本文的序贯目标也按这一机制实现：每一阶段求得最优值后，将其作为下一阶段的附加条件。

因此，CP-SAT 的优势是通用、稳定、自动化程度高；不足是其剪枝主要发生在求解器内部，不容易直接解释为本问题中的某个调度结构。后续自编方法则反过来，显式维护部分调度状态，并由程序主动决定下一步搜索方向。

3.2 融合剪枝的隐枚举法

方法二为融合剪枝的隐枚举法。该方法是在隐枚举搜索框架基础上，结合完工时间下界、能耗下界、机械臂负载下界和中心安全区互斥下界进行剪枝的自编精确搜索方法。其特点是物块数量较少时求解较快、结果可解释；但当物块数量增多时，候选任务和执行模式组合迅速增加，搜索树规模变大，求解速度会明显下降。

3.2.1 决策过程

隐枚举每一步要回答的问题是：在当前部分调度之后，下一个安排哪个任务，以及采用哪一种执行方式。程序从空调度开始，逐步扩展调度序列。每扩展一步时，它枚举“未完成任务-可行执行方式”的组合，并计算该组合加入当前调度后对应的开始时间、结束时间、转移代价和目标值变化。

对一个候选决策的评价分为两层。本章用 Ω 表示一个部分调度状态； $C_{\text{cur}}(\Omega)$ 表示该状态下已安排任务的当前最大完成时刻； $\theta_a(\Omega)$ 表示机械臂 a 在该状态下的可用时间， $\xi_a(\Omega)$ 表示机械臂 a 的当前位置， $\theta_{\text{center}}(\Omega)$ 表示中心区在该状态下的最早释放时间。这些符号只用于描述算法状态，不改变第 2 章模型变量。

第一层是直接评价，即把任务 j 的某个执行模式 $k \in \mathcal{M}_j$ 暂时接到当前部分调度 Ω 后，计算它真实插入后的时间和能耗。候选中机械臂 a 的准备完成时间为

$$R_{ak}(\Omega) = \theta_a(\Omega) + \tau(\xi_a(\Omega), p_j), \quad a \in A_k.$$

双臂任务必须等所有参与机械臂到齐，因此未考虑中心区时的最早同步开始时间为

$$\tilde{S}_k(\Omega) = \max_{a \in A_k} R_{ak}(\Omega).$$

若该任务需要占用中心区，则开始时间继续向后推迟到不与已有中心区区间冲突的位置，记为

$$S_k(\Omega) = \text{NextFree}_{\text{center}}(\tilde{S}_k(\Omega), t_k), \quad E_k(\Omega) = S_k(\Omega) + t_k.$$

若不占用中心区，则 $S_k(\Omega) = \tilde{S}_k(\Omega)$ 。由此可以得到该候选的 **setup** 时间、中心区等待、结束时间、增量能耗以及执行后的机械臂位置。

第二层是前瞻评价。这里的“理想完成”不是构造一条真实调度，而是把剩余问题放松，计算任何真实后续调度都不可能突破的下界。设当前候选加入后的部分调度状态为 Ω ，剩余任务集合为 $\mathcal{R}(\Omega) \subseteq \mathcal{J}$ ，当前最好完整解为

$$Z^{\text{best}} = (C_{\max}^{\text{best}}, E_{\text{tot}}^{\text{best}}, B^{\text{best}}).$$

算法为该候选计算一个乐观目标

$$\underline{Z}(\Omega) = (\underline{C}(\Omega), \underline{E}(\Omega), 0),$$

其中负载均衡项先取 0，表示在前两级目标已经无法改进时才需要继续比较均衡性。若 $\underline{Z}(\Omega)$ 按字典序已经不优于 Z^{best} ，则该候选之后无论怎样安排都不可能得到更好的完整解，可以直接丢弃。

完工时间下界 $\underline{C}(\Omega)$ 由多个放松下界取最大值：

$$\underline{C}(\Omega) = \max\{\underline{C}_{\text{task}}, \underline{C}_{\text{cap}}, \underline{C}_{\text{arm}}, \underline{C}_{\text{center}}\}.$$

各项的计算方式如下：

$$\begin{aligned} \underline{C}_{\text{task}} &= \max\left\{C_{\text{cur}}(\Omega), \max_{j \in \mathcal{R}(\Omega)} \min_{k \in \mathcal{M}_j} E_k(\Omega)\right\}, \\ W_{\text{rem}}(\Omega) &= \sum_{j \in \mathcal{R}(\Omega)} \min_{k \in \mathcal{M}_j} (t_k |A_k| + \rho W_{jk}^{\text{setup}}(\Omega)), \\ \underline{C}_{\text{cap}} &= \max\left\{C_{\text{cur}}(\Omega), \left\lceil \frac{\sum_{a \in \mathcal{A}} \theta_a(\Omega) + W_{\text{rem}}(\Omega)}{|\mathcal{A}|} \right\rceil\right\}, \\ \underline{C}_{\text{arm}} &= \max_{a \in \mathcal{A}} \left\{ \theta_a(\Omega) + \sum_{\substack{j \in \mathcal{R}(\Omega) \\ a \in \cap_{k \in \mathcal{M}_j} A_k}} \min_{k \in \mathcal{M}_j} t_k \right\}, \\ \underline{C}_{\text{center}} &= \max\left\{C_{\text{cur}}(\Omega), \theta_{\text{center}}(\Omega) + \sum_{j \in \mathcal{R}(\Omega)} \min_{k \in \mathcal{M}_j} z_k t_k \right\}. \end{aligned}$$

其中 $W_{jk}^{\text{setup}}(\Omega)$ 为任务 j 在模式 k 下的最小可能 **setup** 工作量， $\rho \in \{0, 1\}$ 表示是否启用 **setup** 容量下界。设

$$\mathcal{P}(\Omega) = \{\xi_a(\Omega) : a \in \mathcal{A}\} \cup \{\text{各收集盒位置}\},$$

则可取

$$W_{jk}^{\text{setup}}(\Omega) = |A_k| \cdot \min_{p \in \mathcal{P}(\Omega)} \left(\frac{\|p - p_j\|_2}{v_0} \cdot \kappa_t \right).$$

这表示：未来某个剩余任务开始前，参与机械臂至少要从当前所在位置或某个已完成任务的收集盒位置移动到该任务起点。由于公式取的是所有可能出发点中距离最短的一个，所以它只会低估真实 **setup**，不会高估。代码中 ρ 采用自适应设置：默认取 0；当启用 **setup** 容量下界开关，或在二臂系统且 $|\mathcal{R}(\Omega)| \geq 4$ 时，取 $\rho = 1$ 。这样设置的原因是，**setup** 容量下界需要为每个剩余任务估计最小空载转移工作量，计算量比只统计处理时间更高；若剩余任务很少，直接继续分支通常更快，额外计算下界未必能带来明显剪枝收益。相反，在二臂系统中，机械臂资源更紧，空载转移会更直接地影响完工时间；当剩余任务不少于 4 个时，后续分支数量开始明显增长，此时把最小 **setup** 工作量加入容量下界，往往能提前排除一批实际不可能优于当前解的候选。

上述四个下界分别对应四种“无论后续怎么排都绕不开”的限制：

- $\underline{C}_{\text{task}}$ 是单任务最早完成下界。 $C_{\text{cur}}(\Omega)$ 是当前部分调度已经达到的完成时间，后续安排不可能把它变小；而 $E_k(\Omega)$ 表示在当前状态下把某个剩余任务作为下一任务执行时的完成时刻。这里要与 $C_{\text{cur}}(\Omega)$ 取最大值，是因为某个剩余任务可能由空闲机械臂很快完成，其 $E_k(\Omega)$ 反而小于当前已经形成的最大完成时刻。
- $\underline{C}_{\text{cap}}$ 是机械臂总容量下界。它不把所有剩余任务串行相加，而是把剩余处理工作量按机械臂数量平均分摊；双臂任务按 $t_k|A_k|$ 计入，是因为它会同时占用两只机械臂。若启用 **setup** 下界，则额外加入最小可能空载转移工作量。
- $\underline{C}_{\text{arm}}$ 是强制机械臂下界。如果某些剩余任务的所有可行模式都必须包含同一只机械臂 a ，这些任务无论如何都要排到该机械臂后面，因此要从 $\theta_a(\Omega)$ 继续累加其最短处理时间。
- $\underline{C}_{\text{center}}$ 是中心区容量下界。中心区是单容量共享资源，同一时刻只能有一个任务组占用；因此对不可避免的中心区占用时间做串行累计。若某任务存在不经过中心区的可行模式，则 $\min_{k \in \mathcal{M}_j} z_k t_k$ 可以取到 0。

能耗下界取

$$\underline{E}(\Omega) = E_{\text{done}}(\Omega) + \sum_{j \in \mathcal{R}(\Omega)} \min_{k \in \mathcal{M}_j} e_k + mE_{\text{sys}},$$

即当前已发生能耗加上每个剩余任务可行模式中的最小处理能耗，再加第 2 章总能耗模型中的系统固定开销 mE_{sys} 。这里的 $E_{\text{done}}(\Omega)$ 不只是已完成任务的处理能耗，还包括已经确定顺序的空载转移能耗：

$$E_{\text{done}}(\Omega) = \sum_{\text{已选 } k} e_k + \sum_{\text{已确定转移 } (u,v)} \epsilon_{u,v}.$$

因此，已经发生的 **setup** 能耗是计入总能耗的。对于尚未安排的剩余任务，未来空载转移路径还没有确定；若强行加入某个具体 **setup** 能耗，可能会高估下界并误剪最优解。因此能耗下界中暂不加入未来 **setup** 能耗，只保留剩余任务最小处理能耗。相应地，未来 **setup** 在时间下界中只通过 $W_{jk}^{\text{setup}}(\Omega)$ 以最小可能值保守加入。

这个过程如图 3 所示。

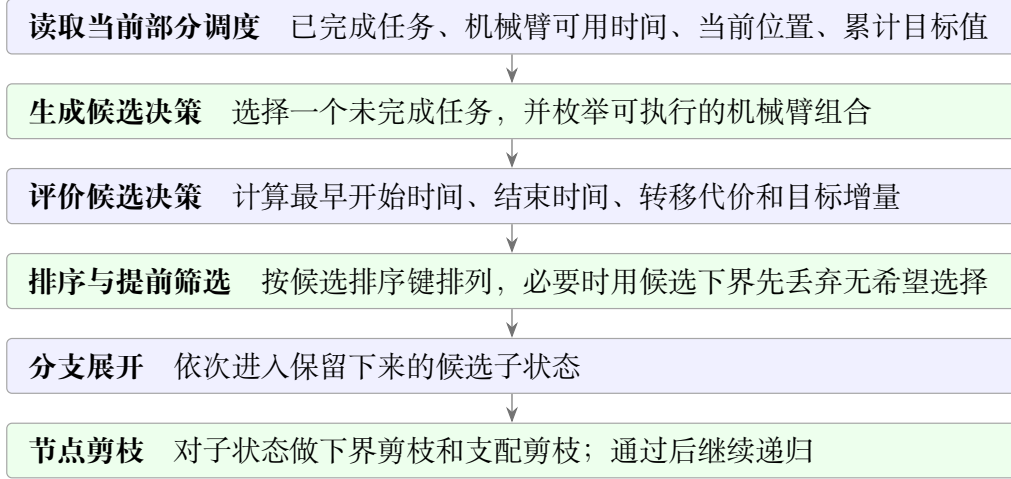


图 3 融合剪枝的隐枚举法精确搜索的决策流程

候选决策不是任意顺序展开，而是按如下排序键展开：

$$\text{key}(\Omega, k) = (C_{\text{cur}}(\Omega'), E_k(\Omega), W_{\text{center}}, E_{\text{tot}}(\Omega'), B(\Omega'), W_k^{\text{setup}}, \Theta_{A_k}(\Omega), j, k),$$

其中 Ω' 为加入候选 k 后的新状态， W_{center} 为中心区等待时间， W_k^{setup} 为该候选的空载转移时间， $\Theta_{A_k}(\Omega) = \max_{a \in A_k} \theta_a(\Omega)$ 。排序只改变搜索顺序，不删除候选方案，因此不影响精确性；它的作用是更早得到较好的完整调度，从而形成更强的当前最好解。

在最终实现中，候选评价还加入了自适应机制：

$$\text{candidate kept} = \begin{cases} \text{true}, & Z^{\text{best}} \text{ 尚不存在,} \\ (C_{\text{max}}(\Omega'), \underline{E}(\Omega'), 0) <_{\text{lex}} Z^{\text{best}}, & \text{轻量筛选阶段,} \\ \underline{Z}(\Omega') <_{\text{lex}} Z^{\text{best}}, & \text{强化筛选阶段.} \end{cases}$$

这里的自适应不是连续调参，而是按问题规模和搜索状态切换。若当前还没有任何完整可行解，即 Z^{best} 尚不存在，算法不会进行下界筛选，因为没有可比较的上界；此时只按排序键选择优先搜索顺序。若已经得到一个完整解，则候选阶段即开始使用下界与 Z^{best} 比较。需要注意的是，小规模算例并非完全不使用能耗下界：代码仍会计算 $\underline{E}(\Omega')$ ，即当前已发生能耗、剩余任务最小处理能耗和系统固定开销之和；只是时间下界只取插入该候选后的当前完工时间 $C_{\text{max}}(\Omega')$ ，不额外计算机械臂容量、强制机械臂、中心区容量等较重的时间下界。当任务数量达到较大规模时，才启用由强化时间下界和能耗下界共同组成的强化筛选。本文实现中以任务数量为主要阈值，当 $n \geq 8$ 且已有当前最好解时启用强化候选筛选；其中 setup 容量下界进一步要求二臂系统且剩余任务数 $|\mathcal{R}(\Omega)| \geq 4$ ，或手动打开 setup 容量下界开关。这一附加条件对应的是收益与开销的平衡：三臂系统的空闲缓冲更大，单个 setup 下界对容量瓶颈的刻画不如二臂系统敏感；而剩余任务少于 4 个时，搜索树已经接近叶节点，直接枚举通常比额外计算 setup 容量下界更划算。这样小规模算例仍保留便宜的能耗筛选，大规模算例又能在搜索树较高层剪去大量无希

望分支。

通过候选阶段筛选的分支会按排序键依次展开。也就是说，算法先把某个候选模式 k 加入当前部分调度，得到子状态 Ω' ；随后递归进入 Ω' ，并在该子状态入口处进行正式的支配剪枝和下界剪枝。若子状态没有被剪去，算法才继续从 Ω' 生成下一层候选。

候选筛选和节点剪枝并不是完全重复。候选筛选发生在“是否进入这个下一步选择”之前，主要用于提前丢弃明显不可能改进当前最好解的候选；而节点剪枝发生在“已经进入某个部分调度状态”之后，判断这个状态是否还值得继续扩展。保留节点剪枝有三点原因：第一，小规模候选阶段只使用轻量时间下界 $C_{\max}(\Omega')$ ，而节点剪枝会重新计算由单任务最早完工、机械臂容量、强制机械臂和中心区容量组成的完整时间下界组合，并配合同一类能耗下界 $E(\Omega')$ ；第二，节点剪枝还包含状态支配判断，这不是候选排序键能替代的；第三，候选列表生成后，前面展开的分支可能找到更好的完整解并更新 Z^{best} ，后续候选虽然在旧上界下通过了筛选，但递归进入子状态时会用更新后的上界再次判断，从而可能被正式剪去。

3.2.2 剪枝决策

隐枚举若完全展开，会面对巨大的组合空间。因此每到一个部分调度，算法都会先判断：在上述放松下界下，这个分支是否仍有可能超过当前最好解。如果答案是否定的，就停止扩展该分支。

本文使用两类正式剪枝。第一类是下界剪枝：利用剩余任务单独最早完工、机械臂容量、强制机械臂、中心区串行占用和最小剩余能耗，估计该分支能达到的最好可能结果。如果该估计已经不优于当前最好解，则剪去。第二类是状态支配剪枝：如果两个部分调度完成的任务相同、机械臂当前位置相同，而其中一个在时间、能耗、负载和中心区释放情况上都不差于另一个，则较差状态没有继续保留的必要。候选生成阶段的提前筛选可以看作正式剪枝前的一道快速过滤，它减少进入递归的候选数量；节点剪枝则是搜索树上的保守判定，保证每个真正进入的部分调度状态仍然满足继续扩展的必要性。

融合剪枝的隐枚举法的优点是搜索过程可解释性强，在完整搜索结束时可以给出全局最优性保证；不足是任务规模较大时，即使有剪枝，也可能需要较长时间。

3.3 启发式算法

方法三为启发式算法。该方法不是课内某个标准算法的直接套用，而是根据本项目多机械臂搬运任务的结构自主设计，用于在较短时间内构造质量较好的可行调度方案。

3.3.1 初始解构造

启发式首先从多个角度构造初始调度。不同策略优先考虑不同因素，例如尽快完成任务、减少中心区等待、降低能耗或改善机械臂负载均衡。这样做是为了避免单一贪心规则过早固定某一种任务顺序。

在构造过程中，算法每次从未安排任务中挑选一个候选任务及其执行方式，并立即把它插入当前调度。局部评分采用简化前瞻形式：

$$H(\Omega') = (\hat{C}(\Omega'), C_{\text{cur}}(\Omega'), E_{\text{done}}(\Omega') + E_{\text{rem}}(\Omega'), B(\Omega')),$$

其中 Ω' 为插入后的状态。这里的“简化前瞻”可以理解为：不完整求解剩余调度，只用很便宜的乐观估计判断这个插入动作是否会给后续留下过重负担。具体地，令

$$D_{\text{rem}}(\Omega') = \sum_{j \in \mathcal{R}(\Omega')} \min_{k \in \mathcal{M}_j} t_k, \quad E_{\text{rem}}(\Omega') = \sum_{j \in \mathcal{R}(\Omega')} \min_{k \in \mathcal{M}_j} e_k.$$

其中 $D_{\text{rem}}(\Omega')$ 表示剩余任务至少还需要的处理时间总量， $E_{\text{rem}}(\Omega')$ 表示剩余任务至少还会产生的处理能耗。算法进一步把剩余处理时间按机械臂数量平均分摊，得到乐观完工估计

$$\hat{C}(\Omega') = \max \left\{ C_{\text{cur}}(\Omega'), \left\lceil \frac{\sum_{a \in \mathcal{A}} \theta_a(\Omega') + D_{\text{rem}}(\Omega')}{|\mathcal{A}|} \right\rceil \right\}.$$

因此， $H(\Omega')$ 的含义是按顺序比较四件事：该插入后从平均负载角度看未来最早可能多晚完成、当前已经形成的最大完成时间、当前能耗加剩余最小能耗、以及机械臂负载差。该评分不是用于证明最优的严格下界，而是用于排序候选决策；它比只看当前任务结束时间更稳，因为它会惩罚那些当前完成很快、但把大量剩余工作集中留给后续的选择。多个初始解生成后，算法保留字典序目标最好的一个作为后续改进起点。

为了进一步提高初始解质量，启发式中还加入了限宽 **beam search**。它不是只保留一条当前最优的部分调度，而是在每一层保留若干条评分靠前的部分调度路径；下一层再从这些路径继续扩展。**beam** 中部分调度的排序键为

$$(\hat{C}(\Omega), C_{\text{cur}}(\Omega), E_{\text{done}}(\Omega) + E_{\text{rem}}(\Omega), B(\Omega), |\mathcal{R}(\Omega)|),$$

其中 $\hat{C}(\Omega)$ 用当前各机械臂可用时间和剩余最短处理时间估计未来平均负载后的乐观完工时间， $E_{\text{rem}}(\Omega)$ 是剩余任务最小能耗之和。该排序使 **beam search** 保留的不是“当前看起来最早”的单一路径，而是兼顾未来工作量、能耗和负载均衡的多条路径。**beam** 宽度根据任务数量和机械臂数量自适应调整：规模较小时保留更多路径以提高解质量，规模较大或三臂组合较多时适当收窄宽度以控制计算时间。这样比单一路径贪心更稳，又比完整枚举快得多。

3.3.2 大邻域改进

得到初始解后，算法进入大邻域搜索。一次迭代包括“破坏”和“修复”两个动作。破坏阶段从当前完整调度中移除一部分任务，被移除的任务可以随机选取，也可以优先选择造成等待、结束较晚或涉及双臂协同的任务。

修复阶段采用逐步最优插入。设破坏后留下的任务序列为基准序列，被移除任务集合为 $\mathcal{D}$ 。修复的目标不是把 $\mathcal{D}$ 中的任务放回原来的位置，而是在当前局部邻域内重新决定这些任务的执行方式和相对位置。算法每次只固定一个插入决策，具体过程为：

1. 对每个尚未插回的任务 $j \in \mathcal{D}$ ，枚举其所有可行模式 $k \in \mathcal{M}_j$ ；
2. 对每个模式，枚举它可以插入基准序列的所有位置，包括最前端、任意两个任务之间以及最后端；

3. 对每一种插入方案，将新的任务序列从头顺序解码，得到完整调度，并计算其 $(C_{\max}, E_{\text{tot}}, B)$;
4. 选择字典序目标最小的插入方案；若 $(C_{\max}, E_{\text{tot}}, B)$ 相同，则继续用中心区等待时间、 setup 时间和任务编号等规则打破并列；
5. 将该任务固定在选定位置，并对剩余被移除任务重复上述过程，直到 $\mathcal{D}$ 为空。

这里需要重新解码整条序列，是因为一个任务插入到某个位置后，插入点之后的任务开始时间、机械臂当前位置、 setup 时间和中心区等待都可能连锁变化，不能只评价被插入任务本身。但这种重算不会使算法退化为完整枚举，原因有三点。第一，枚举只发生在被破坏的小邻域内。设任务总数为 n ，代码中每轮破坏的基准规模为

$$q_0 = \min \left\{ \max \left(2, \left\lfloor \frac{n}{3} \right\rfloor \right), n - 1 \right\},$$

实际迭代时再在 $q_0 - 1, q_0, q_0 + 1$ 附近小幅变化，并始终限制在 $[1, n - 1]$ 内，因此不会把全部任务重新排列。第二，修复采用逐个插回的贪心策略：一次插入决策的候选数量大致为

$$|\mathcal{D}_{\text{rem}}| \times \text{平均可行模式数} \times (\text{当前序列长度} + 1),$$

选出当前最好的插入后立即固定，再处理下一个任务；算法并不会同时枚举 $\mathcal{D}$ 中所有任务的排列、模式组合和位置组合。第三，大邻域搜索本身还受到时间限制、最大迭代次数和连续未改进次数限制。因此，该步骤本质上是“局部枚举 + 贪心修复”，用有限计算换取较好的邻域改进质量，而不是全局完整枚举。

这种做法比简单交换两个相邻任务更灵活。多机械臂调度中，一个任务位置改变可能同时影响多只机械臂的后续可用时间；双臂协同任务尤其明显，一次改变会联动两只机械臂。大邻域搜索通过一次移除多个任务，给算法留下更大的重排空间，更容易跳出局部最优。

新调度生成后，算法按字典序目标判断是否接受。若新解更好，则直接接受；在搜索前期，也允许少量不明显更差的解进入下一轮，以避免过早停在局部最优；搜索后期则更偏向稳定改进，并尝试在不增加最大完工时间的前提下降低能耗。

大邻域搜索结束后，算法还会进行一轮末端能耗精化。由于本文目标是序贯的，最大完工时间优先级最高，因此该阶段只接受不增大 $C_{\max}$ 的局部调整。具体而言，算法依次尝试三类小改动：为单个任务更换可行模式、将单个任务移动到其他位置、交换两个任务的位置。若某个改动保持 $C_{\max}$ 不变或更小，并且能降低能耗或负载差，则接受该改动并继续搜索。这个阶段不再追求大范围重排，而是在已有较好调度附近压低次级目标。

启发式的时间分配也采用自适应策略。总时间被分给 **beam search**、大邻域搜索和末端能耗精化三个阶段：任务较少时，候选空间相对有限，算法只给 **beam search** 和大邻域搜索较短时间，并保留少量时间做末端能耗精化；任务较多时，更多时间分配给 **beam search** 和大邻域搜索，以便先找到结构较好的任务顺序，再在末端对能耗进行小范围修正。大邻域搜索中的破坏规模采用上述 q_0 规则，核心含义是每轮大约重排三分之一的

任务，但至少移除 2 个、至多保留 1 个任务不被移除，并在相邻迭代中做小幅扰动。这样每次重排既能改变足够多的任务，又不会让修复过程变得过慢。这个自适应组合是启发式效率提升的关键：它避免了“小问题上搜索策略过重、大问题上探索不足”的两种低效情况。

3.4 热启动后的隐枚举法

方法四为热启动后的隐枚举法。该方法将启发式算法与融合剪枝的隐枚举法结合：先由启发式算法快速生成一个较优完整可行解，并将其作为初始上界；随后融合剪枝的隐枚举法从空调度开始继续搜索，利用该初始上界提前剪去不可能优于当前最好解的分支。

热启动阶段本身采用自适应策略。设任务总数为 n ，机械臂数量为 m 。当

$$n \geq 8, \quad m \geq 3$$

时，代码采用完整启发式热启动；否则采用轻量热启动。两种路径的共同点是：启发式只负责给出一个完整可行调度 S^{init} ，随后隐枚举仍从空调度开始搜索，并把 S^{init} 的目标值作为初始上界

$$Z_0^{\text{best}} = (C_{\max}(S^{\text{init}}), E_{\text{tot}}(S^{\text{init}}), B(S^{\text{init}})).$$

因此，热启动不会改变可行域、候选生成方式、下界剪枝规则或状态支配规则；它改变的是隐枚举一开始用于比较的 **incumbent**。

轻量热启动用于小规模算例或二臂算例，其流程为

$$\begin{aligned} &\text{四个贪心完整解} \rightarrow S^{\text{greedy}} \rightarrow \text{限时0.20 s 的 beam search} \rightarrow S^{\text{beam}}, \\ &S^{\text{init}} = \arg \min_{\text{lex}} \{S^{\text{greedy}}, S^{\text{beam}}\}. \end{aligned}$$

其中 S^{greedy} 是四种贪心策略中目标值最好的完整调度； S^{beam} 是 beam search 从空调度独立构造出的完整调度，而不是在贪心解上继续修改。若 beam search 在时间限制内没有返回完整解，则直接使用 S^{greedy} 。这一路径的目的不是深度改进，而是用很小的预处理时间获得一个不差的初始上界。

完整启发式热启动用于 $n \geq 8, m \geq 3$ 的场景，其流程为

$$\begin{aligned} &\text{beam search 预热} \rightarrow S^{\text{beam}}, \\ &\text{四个贪心完整解} \rightarrow S^{\text{greedy}}, \\ &\min_{\text{lex}} \{S^{\text{beam}}, S^{\text{greedy}}\} \rightarrow \text{多启动大邻域搜索} \rightarrow \text{同 } C_{\max} \text{ 能耗精化} \rightarrow S^{\text{init}}. \end{aligned}$$

这里的 beam search 和四个贪心解同样是两个候选来源，代码先分别得到可行调度，再按字典序目标取更优者作为大邻域搜索的起点。随后 LNS 通过破坏-修复在局部邻域中改进任务顺序和执行模式；末端能耗精化只接受不增大 $C_{\max}$ 的调整，用于压低能耗或负载差。该路径只在大规模三臂场景启用，是因为此时机械臂组合更多、搜索树更大，较好的初始上界更容易转化为后续隐枚举的剪枝收益。

获得 S^{init} 后，隐枚举的执行逻辑保持为：

1. 将 Z_0^{best} 作为搜索开始时的当前最好目标值；
2. 从空调度出发，按前述候选生成、候选筛选、节点剪枝和状态支配规则完整搜索；
3. 搜索过程中若发现更好的完整调度，则更新 Z^{best} ，后续分支使用更强上界继续剪枝；
4. 若隐枚举搜索完成，则最终结果仍由分支定界证明最优。

该方法的亮点有三点。第一，它保留了精确算法的最优性证明：启发式解只作为初始 *incumbent*，不会改变任何可行解，也不会改变安全下界。第二，它把启发式的优势转化为剪枝优势：初始上界越好，越多分支会因为下界不可能优于 Z^{best} 而被提前剪去。第三，它根据问题规模选择 *warm start* 强度，避免“小问题上预处理过重、大问题上初始上界过弱”的情况。因此，热启动后的隐枚举法兼具启发式的快速找解能力和隐枚举的全局最优性证明能力。

3.5 二臂/三臂模式选择在主问题中的实现

二臂和三臂系统的差异，首先体现在可行模式集合 $\mathcal{M}_j$ 的构造上。对每个任务 j ，代码并不是在搜索过程中临时判断“哪只机械臂能做”，而是在调度前先生成所有可行执行模式：

$$k = (j, A_k, t_k, e_k, z_k) \in \mathcal{M}_j.$$

其中 A_k 是该模式占用的机械臂集合， t_k 、 e_k 和 z_k 分别为该模式的处理时间、处理能耗和中心区标记。

具体来说，Type1 和 Type2 为单臂任务，代码会检查每只机械臂是否同时能够到达任务起点 p_j 和目标收集盒 q_j ；若机械臂 a 可达，则生成一个单臂模式

$$A_k = \{a\}.$$

Type3 和 Type4 为双臂协同任务，代码会枚举所有大小为 2 的机械臂组合，并保留组合中两只机械臂都能到达 p_j 和 q_j 的模式：

$$A_k = \{a, b\}, \quad a \neq b.$$

因此，在二臂系统中，双臂任务最多只有一个协同组合 $\{1, 2\}$ ；在三臂系统中，同一个双臂任务可能产生三个候选组合 $\{1, 2\}$ 、 $\{1, 3\}$ 和 $\{2, 3\}$ 。这意味着三臂系统不仅要决定任务顺序，还要决定每个双臂任务由哪一对机械臂完成。

在隐枚举和启发式搜索中，模式选择直接进入候选决策层。算法遍历所有尚未完成任务 $j \in \mathcal{R}(\Omega)$ 及其所有模式 $k \in \mathcal{M}_j$ ，因此“选择任务”和“选择机械臂组合”是在同一步完成的。每个候选模式的时间、*setup*、中心区等待、能耗和负载变化，仍按前文候选决策评价方法计算，此处不再重复展开。

因此，三臂模式选择的本质不是简单增加一只机械臂，而是让每个双臂任务多出“选择哪两只机械臂”的组合决策。例如，某个组合当前 setup 较短，可能使该任务很快完成；但如果它占用了后续单臂任务急需的机械臂，负载差和后续容量下界会变差。另一个组合当前稍慢，却可能释放一只关键机械臂，使后续任务更均衡地展开。自编算法正是通过对每个候选模式进行上述显式评价，在搜索过程中比较这些取舍。

这种处理方式与 CP-SAT 的内部搜索不同。CP-SAT 将不同机械臂组合表示为若干可选区间，由求解器通过传播和分支自动决定是否选择；自编算法则把机械臂组合作为候选决策的一部分显式展开、评分和排序。这样做的好处是可解释性更强：当三臂方案优于二臂方案时，通常可以从同步等待减少、中心区等待减少、 setup 缩短或负载更均衡等方面看出原因。

3.6 与非线性扩展的衔接

前文已经分别在二臂系统和三臂系统下完成了主调度问题的求解，得到了每组任务实例对应的最优最大完工时间 $C_{\max}$ 和系统总能耗 E_{tot} 。为了进一步判断在主问题结果下应选择二臂模式还是三臂模式，对二臂与三臂的最优调度结果进行统一整理和对比。

该分析并不重新求解调度模型，而是以主问题已经得到的二臂、三臂最优结果为输入，进行结果层面的模式选择。由于不同 seed 代表同一任务规模下不同的随机空间分布，单个 seed 的结果可能受到物块位置影响，因此程序对多个 seed 取平均，以减弱随机生成位置带来的偶然性。

在每个 seed 下，程序计算三臂相对二臂的时间节省率：

$$\eta_T = \frac{C_{\max}^{2B} - C_{\max}^{3B}}{C_{\max}^{2B}} \times 100$$

当 $\eta_T > 0$ 时，说明三臂模式比二臂模式更快，具有完成时间优势。

同时，程序计算三臂相对二臂的能耗增加率：

$$\eta_E = \frac{E_{\text{tot}}^{3B} - E_{\text{tot}}^{2B}}{E_{\text{tot}}^{2B}} \times 100$$

当 $\eta_E > 0$ 时，说明三臂模式虽然缩短了完成时间，但带来了额外能耗代价。

为了把时间收益和能耗代价放在同一评价框架下，程序进一步构造综合收益指标：

$$S_\lambda = \eta_T - \lambda \eta_E.$$

其中， λ 为时间-能耗偏好系数。 λ 越小，表示决策者越重视完成时间； λ 越大，表示决策者越重视能耗。当 $S_\lambda > 0$ 时，说明三臂带来的时间收益足以抵消能耗增加，推荐三臂模式；当 $S_\lambda < 0$ 时，说明能耗代价在该偏好下更重要，推荐二臂模式。

由此还可以得到临界偏好系数：

$$\lambda^* = \frac{\eta_T}{\eta_E}.$$

当 $\lambda < \lambda^*$ 时，三臂模式更占优；当 $\lambda > \lambda^*$ 时，二臂模式更占优。因此， λ^* 可以理解为二臂和三臂推荐结果发生转换的分界点。

在具体实现中，程序设置了多组偏好系数：

$$\lambda = 0.5, 1, 2, 3, 4, 5, 6, 8, 10, 15.$$

程序分别计算每组任务组合下的 S_λ 和推荐模式。

从统计结果看，三臂模式在所有任务组合中都能够缩短最大完工时间。16 组任务组合的平均结果显示，二臂模式平均最大完工时间为 2554.17，三臂模式平均最大完工时间为 1758.69，三臂相对二臂平均节省约 31.24% 的完成时间。这说明三臂系统由于增加了可选机械臂组合和并行执行能力，确实能够显著改善主问题中的时间目标。

但是，三臂模式并不是无条件更优。从能耗角度看，二臂模式平均能耗为 11479.25，三臂模式平均能耗为 11976.56，三臂相对二臂平均增加约 4.46% 的能耗。这说明第三只机械臂虽然提高了并行能力，但也带来了系统固定能耗和协调代价。因此，如果只考虑完成时间，16 组任务组合均推荐三臂；如果只考虑能耗，16 组任务组合均推荐二臂。

进一步观察不同 λ 下的推荐结果可以发现：当 $\lambda = 0.5$ 到 $\lambda = 5$ 时，所有任务组合均推荐三臂，说明在时间优先或时间、能耗相对均衡的情形下，三臂的时间收益仍然占主导；当 $\lambda = 6$ 时，已有部分任务组合转为推荐二臂；当 $\lambda = 8$ 时，多数任务组合转为推荐二臂；当 $\lambda = 10$ 和 $\lambda = 15$ 时，所有任务组合均推荐二臂。这表明随着能耗权重增大，三臂模式的额外能耗逐渐抵消其时间优势，最终二臂模式成为更合适的选择。

综上，主问题下的二臂/三臂模式选择不是简单判断机械臂数量越多越好，而是一个时间收益与能耗代价之间的权衡问题。三臂模式的主要优势在于缩短最大完工时间，适合工期紧张、并行需求强的任务场景；二臂模式的主要优势在于系统能耗较低，适合时间约束相对宽松、能耗优先的任务场景。该分析也为后续非线性速度优化提供了基础：主问题先确定二臂和三臂在基准速度下的最优调度表现，后续再进一步讨论在改变速度倍率后，这种二臂/三臂优劣关系是否会发生变化。

4 延伸问题：非线性规划

机械臂运行速度提高时，单个任务的搬运时间会缩短，系统最大完工时间 $C_{\max}$ 可能下降；但较高速度通常也会带来更大的驱动能耗、协调能耗和控制代价。相反，若机械臂运行速度降低，系统能耗可能下降，但任务持续时间会变长，甚至可能无法满足给定的截止时间要求。

这种处理方式可以理解为先离散调度，后连续调速。前一层由整数规划、目标规划和隐枚举等方法确定调度结构，后一层在调度结构已知的条件下，通过非线性规划优化速度参数。既避免了将离散变量和连续变量完全耦合所带来的求解困难，又能够进一步分析速度、完成时间和能耗之间的权衡关系。

主问题已经给出了某一系统模式下的基准调度结果，仅将速度倍率作为连续决策变量。

设单臂任务速度倍率为 s_1 ，双臂协同任务速度倍率为 s_2 ，二者满足速度上下界约

束：

$$s_{\min} \leq s_1 \leq s_{\max}, \quad s_{\min} \leq s_2 \leq s_{\max}.$$

其中， $s_1 = 1$ 和 $s_2 = 1$ 表示采用主问题中的基准速度； $s_1 > 1$ 或 $s_2 > 1$ 表示提高对应任务类型的运行速度； $s_1 < 1$ 或 $s_2 < 1$ 表示降低对应任务类型的运行速度。

在调度结构已知的情况下，系统最大完工时间可以表示为速度倍率的函数。由于任务执行时间与速度近似成反比，因此可写为：

$$C_{\max}(s_1, s_2) = C_0 + \frac{T_1}{s_1} + \frac{T_2}{s_2}.$$

其中， C_0 表示与速度倍率无关的固定时间部分， T_1 表示单臂任务对应的可压缩时间部分， T_2 表示双臂协同任务对应的可压缩时间部分。

能耗函数则表示为速度倍率的非线性函数：

$$E(s_1, s_2) = E_0 + a_1 s_1^\alpha + a_2 s_2^\beta + \rho_1 (s_1 - 1)^2 + \rho_2 (s_2 - 1)^2.$$

其中， E_0 表示与速度倍率无关的固定能耗部分， $a_1 s_1^\alpha$ 和 $a_2 s_2^\beta$ 分别表示单臂任务和双臂协同任务的速度相关能耗， α 和 β 为非线性指数， $\rho_1 (s_1 - 1)^2$ 和 $\rho_2 (s_2 - 1)^2$ 表示速度偏离基准速度时带来的附加代价。

在给定截止时间 D 的条件下，两个延伸问题都需要满足：

$$C_{\max}(s_1, s_2) \leq D.$$

因此，共同的非线性优化思想可以概括为：

$$\min_{s_1, s_2} E(s_1, s_2)$$

$$\text{s.t. } C_{\max}(s_1, s_2) \leq D,$$

$$s_{\min} \leq s_1 \leq s_{\max}, \quad s_{\min} \leq s_2 \leq s_{\max}.$$

上述模型说明，本章非线性规划部分的核心不再是选择哪只机械臂执行任务，而是在调度方案已经确定之后，进一步选择合适的运行速度。后续两个问题将在这一共同模型基础上分别进行扩展。

4.1 问题一：速度-能耗非线性优化问题

4.1.1 问题分析

不同 seed 对应不同的物块空间分布，因此会导致搬运距离、任务衔接关系、最大完工时间和系统能耗产生一定波动。若直接选取某一个 seed 进行速度优化，结果可能具有较强的偶然性，不能代表该任务规模下的整体表现。因此首先对多个 seed 的主问题结果进行平均，得到二臂和三臂模式下的平均基准完工时间与平均基准能耗。

设某一模式下的平均基准最大完工时间为 $C_{\max}^0$ ，平均基准能耗为 E^0 。根据任务类

型数量将基准时间和基准能耗拆分为单臂任务部分和双臂协同任务部分。

设 Type1 和 Type2 的任务数量分别为 n_1, n_2 , Type3 和 Type4 的任务数量分别为 n_3, n_4 。单臂任务占比和双臂任务占比分别定义为

$$\omega_1 = \frac{n_1 + n_2}{n_1 + n_2 + n_3 + n_4}, \quad \omega_2 = \frac{n_3 + n_4}{n_1 + n_2 + n_3 + n_4}.$$

于是基准完工时间可近似拆分为

$$T_1 = \omega_1 C_{\max}^0, \quad T_2 = \omega_2 C_{\max}^0.$$

其中, T_1 表示单臂任务对应的可压缩时间部分, T_2 表示双臂协同任务对应的可压缩时间部分。

同理, 基准能耗可拆分为

$$E_1^0 = \omega_1 E^0, \quad E_2^0 = \omega_2 E^0.$$

其中, E_1^0 表示单臂任务对应的基准能耗, E_2^0 表示双臂协同任务对应的基准能耗。

在此基础上, 引入单臂任务速度倍率 s_1 和双臂协同任务速度倍率 s_2 。当速度倍率改变后, 最大完工时间表示为

$$C_{\max}(s_1, s_2) = \frac{T_1}{s_1} + \frac{T_2}{s_2}.$$

该式表示速度倍率越大, 对应任务类型的执行时间越短。

能耗模型:

$$E(s_1, s_2) = E_1^0 \left[\frac{\lambda_1}{s_1} + (1 - \lambda_1) ((1 - \rho_1) + \rho_1 s_1^2) \right] + E_2^0 \left[\frac{\lambda_2}{s_2} + (1 - \lambda_2) ((1 - \rho_2) + \rho_2 s_2^2) \right].$$

其中, λ_1, λ_2 用于刻画低速运行时因任务持续时间增加而产生的保持能耗影响, ρ_1, ρ_2 用于刻画高速运行时驱动能耗增加的影响。该模型并不是简单认为速度越低能耗越小, 而是认为存在一个相对经济的速度区间: 速度过低会拉长执行时间, 速度过高会增加驱动代价。

对于给定截止时间 D , 问题一最终可写为

$$\begin{aligned} & \min_{s_1, s_2} E(s_1, s_2) \\ & \text{s.t. } C_{\max}(s_1, s_2) \leq D, \\ & s_{\min} \leq s_1 \leq s_{\max}, \quad s_{\min} \leq s_2 \leq s_{\max}. \end{aligned}$$

其中, 截止时间 D 由基准完工时间乘以不同截止时间比例得到。通过改变截止时间比例, 可以观察在宽松工期和紧张工期下, 二臂模式与三臂模式的最优速度、最小能耗

和推荐结果如何变化。因此，问题一的核心不是重新安排任务，而是在平均调度结果已经确定的条件下，进一步回答“在不同工期要求下，应该以多快的速度运行才更节能”。

4.1.2 实现过程

本问题属于带不等式约束的连续非线性规划问题，决策变量为单臂任务速度倍率 s_1 和双臂协同任务速度倍率 s_2 。由于速度倍率必须为正，且存在上下界约束和截止时间约束，因此本文采用内点法处理约束，并在每次确定下降方向后使用 0.618 法进行一维步长搜索。

首先说明该问题满足使用上述方法的基本条件。根据前文建模，系统完工时间函数可写为

$$C_{\max}(s_1, s_2) = \frac{T_1}{s_1} + \frac{T_2}{s_2}.$$

在 $s_1 > 0, s_2 > 0$ 的定义域内，函数 $1/s$ 是凸函数，因此 $C_{\max}(s_1, s_2)$ 也是凸函数。截止时间约束

$$C_{\max}(s_1, s_2) \leq D$$

对应凸函数的下水平集，因此形成凸可行域。

能耗函数采用 U 形经济速度模型。对任一任务类型，其速度相关能耗项可写为

$$f_i(s) = \frac{\lambda_i}{s} + (1 - \lambda_i) [(1 - \rho_i) + \rho_i s^2], \quad i = 1, 2.$$

对其求二阶导数，有

$$f_i''(s) = \frac{2\lambda_i}{s^3} + 2(1 - \lambda_i)\rho_i.$$

由于 $s > 0$ ，且 λ_i, ρ_i 均为非负参数，因此

$$f_i''(s) \geq 0.$$

这说明单个速度能耗项为凸函数。总能耗函数由若干凸函数和常数项加权相加得到，因此仍为凸函数。由此，本问题可以看作在凸可行域上最小化凸目标函数的非线性规划问题。该性质保证了局部最优解具有较好的全局解释，也使得内点法和一维搜索具有合理的使用基础。

为使用内点法，将所有不等式约束统一写成 $g_i(s_1, s_2) > 0$ 的形式。具体定义为

$$g_0(s_1, s_2) = D - C_{\max}(s_1, s_2),$$

$$g_1(s_1, s_2) = s_1 - s_{\min}, \quad g_2(s_1, s_2) = s_{\max} - s_1,$$

$$g_3(s_1, s_2) = s_2 - s_{\min}, \quad g_4(s_1, s_2) = s_{\max} - s_2.$$

其中， g_0 对应截止时间约束， g_1 到 g_4 对应两个速度变量的上下界约束。只要所有 $g_i > 0$ ，当前点就严格位于可行域内部。

内点法的核心思想是将原来的约束优化问题转化为无约束优化问题。本文构造如下

对数障碍函数：

$$\Phi_{\mu}(s_1, s_2) = E(s_1, s_2) - \mu \sum_{i=0}^4 \ln g_i(s_1, s_2),$$

其中， $\mu > 0$ 为障碍参数。由于 $\ln g_i$ 只有在 $g_i > 0$ 时才有定义，因此迭代点会被限制在可行域内部。当某个约束即将被违反时，对应的 g_i 接近 0， $-\ln g_i$ 会迅速增大，从而阻止迭代点越过可行域边界。随着 μ 逐渐减小，障碍项影响减弱，障碍函数的最优点逐渐逼近原约束问题的最优解。

在每一次内点迭代中，程序先根据当前速度点

$$\mathbf{s}^{(k)} = \begin{bmatrix} s_1^{(k)} & s_2^{(k)} \end{bmatrix}$$

计算障碍函数的梯度 $\nabla \Phi_{\mu}(\mathbf{s}^{(k)})$ ，并取负梯度方向作为下降方向：

$$\mathbf{d}^{(k)} = -\nabla \Phi_{\mu}(\mathbf{s}^{(k)}).$$

这样，原来的二维搜索问题就转化为沿着方向 $\mathbf{d}^{(k)}$ 寻找合适步长的问题。新的速度点表示为

$$\mathbf{s}^{(k+1)} = \mathbf{s}^{(k)} + \alpha \mathbf{d}^{(k)},$$

其中 α 是待确定的一维步长。

0.618 法本身是一维搜索方法，不能直接用于二维变量 (s_1, s_2) 。本文的处理方式是：先由内点法确定二维空间中的下降方向 $\mathbf{d}^{(k)}$ ，再只在该方向上搜索步长 α 。于是构造一维函数

$$\varphi(\alpha) = \Phi_{\mu}(\mathbf{s}^{(k)} + \alpha \mathbf{d}^{(k)}).$$

此时， $\varphi(\alpha)$ 只含一个变量 α ，可以使用 0.618 法进行求解。

由于障碍函数 Φ_{μ} 在可行域内部保持凸性，而凸函数沿任意直线方向的限制函数仍为一维凸函数，因此 $\varphi(\alpha)$ 在可行步长区间内具有单峰性。0.618 法正适用于这种一维单峰函数的最小值搜索。因此，本文不是直接用 0.618 法求二维最优点，而是将二维优化转化为“方向确定 + 步长搜索”的形式，其中 0.618 法只用于确定每一步的最优步长。

在实际实现中，首先根据速度上下界和截止时间约束确定可行步长区间

$$0 \leq \alpha \leq \alpha_{\max},$$

并保证对任意候选步长都有

$$g_i(\mathbf{s}^{(k)} + \alpha \mathbf{d}^{(k)}) > 0, \quad i = 0, 1, \dots, 4.$$

然后在该区间内使用 0.618 法不断缩小搜索范围，比较两个试探点处的 $\varphi(\alpha)$ 值，保留函数值较小的一侧区间，直到区间长度小于给定精度为止。最终得到当前方向上的近似最优步长 $\alpha^{(k)}$ 。

这个过程如图 4 所示。



图 4 内点法结合 0.618 法求解速度-能耗非线性优化问题的流程

通过上述方法，程序最终得到最优单臂速度倍率 s_1^* 、最优双臂速度倍率 s_2^* 、优化后的最大完工时间 $C_{\max}(s_1^*, s_2^*)$ 和优化后的总能耗 $E(s_1^*, s_2^*)$ 。该求解过程体现了内点法处理不等式约束、0.618 法处理一维步长搜索的结合：前者保证迭代始终在可行域内部进行，后者保证每一步沿下降方向取得较优步长，从而完成二维非线性规划问题的数值求解。

4.1.3 KKT 验证条件

对于速度-能耗非线性优化问题，其标准形式可以写为

$$\begin{aligned} \min_x \quad & f(x) \\ \text{s.t.} \quad & g_j(x) \geq 0, \quad j = 1, 2, \dots, 5. \end{aligned}$$

其中， $x = (s_S, s_D)^T$ ， g_1 至 g_4 分别对应速度上下界约束， g_5 对应截止时间约束。

该问题的 KKT 条件为

$$\begin{aligned} \nabla f(x^*) - \sum_{j=1}^5 \mu_j \nabla g_j(x^*) &= 0, \\ g_j(x^*) &\geq 0, \quad j = 1, 2, \dots, 5, \end{aligned}$$

$$\begin{aligned}\mu_j &\geq 0, & j = 1, 2, \dots, 5, \\ \mu_j g_j(x^*) &= 0, & j = 1, 2, \dots, 5.\end{aligned}$$

其中，第一式为平稳性条件，第二式为原问题可行性条件，第三式为乘子非负条件，第四式为互补松弛条件。

本文对速度-能耗优化结果逐行检查上述条件。具体检查内容包括：各约束裕量 g_1, g_2, g_3, g_4, g_5 是否满足非负约束；对应乘子 $\mu_1, \mu_2, \mu_3, \mu_4, \mu_5$ 是否满足非负条件；平稳性残差

$$\left\| \nabla f(x^*) - \sum_{j=1}^5 \mu_j \nabla g_j(x^*) \right\|$$

是否足够小；以及最大互补误差

$$\max_j |\mu_j g_j(x^*)|$$

是否在数值容差范围内。

4.2 问题二：鲁棒速度模式选择问题

4.2.1 问题分析

问题二是在问题一基础上的进一步扩展。问题一对同一任务数量组合下的多个 seed 结果取平均，得到平均意义下的速度-能耗优化结果。这种处理方式能够反映某一任务规模的典型表现，但也会掩盖不同随机 seed 之间的差异。实际上，不同 seed 对应不同的物块空间分布，物块到收集盒的距离、任务之间的空载转移代价、中心安全区占用情况以及机械臂负载情况都会发生变化。因此，即使任务数量组合相同，不同 seed 下的基准完工时间和基准能耗也可能不同。

如果只根据平均结果选择速度倍率，可能会出现一种情况：该速度在平均意义下满足截止时间约束，但对于某些较困难的随机实例并不可行，或者在某些 seed 下能耗明显偏高。因此，问题二不再把多个 seed 简单平均，而是保留每个 seed 对应的独立调度结果，并要求同一任务数量组合下的所有 seed 共用同一组速度倍率。这样得到的速度方案不只适用于某一个随机实例，而是对同一任务规模下的多种空间分布都具有可行性。

该问题可以理解为鲁棒速度模式选择问题。这里的“鲁棒”主要体现在两个方面：第一，速度方案必须使所有 seed 都满足截止时间约束，而不是只满足平均情况；第二，优化目标不再是平均能耗最小，而是最坏情形能耗尽可能小。也就是说，模型关注的是在随机位置扰动下，二臂或三臂模式是否仍然能够稳定地满足工期要求，并控制最不利情况下的能耗水平。

设同一任务数量组合下的 seed 集合为 $\mathcal{R}$ 。对于任意 seed $r \in \mathcal{R}$ ，主问题已经分别给出了二臂或三臂模式下的基准调度结果。与问题一类似，本问题仍然不改变该 seed 下的任务分配和执行顺序，只对单臂任务速度倍率和双臂协同任务速度倍率进行优化。设单臂任务速度倍率为 s_S ，双臂协同任务速度倍率为 s_D ，其中下标 S 表示 single-arm，下标 D 表示 dual-arm。

对于每一个 seed r ，根据该 seed 的主问题调度结果，可以得到单臂任务对应的可压缩时间 $T_S^{(r)}$ 、双臂协同任务对应的可压缩时间 $T_D^{(r)}$ 。因此，该 seed 下速度调整后的最大完工时间可表示为

$$C_{\max}^{(r)}(s_S, s_D) = \frac{T_S^{(r)}}{s_S} + \frac{T_D^{(r)}}{s_D}.$$

其中， $T_S^{(r)}$ 和 $T_D^{(r)}$ 随 seed 改变而改变，反映不同物块空间分布对任务时间结构的影响。

类似地，设 seed r 下单臂任务基准能耗为 $E_S^{0,(r)}$ ，双臂协同任务基准能耗为 $E_D^{0,(r)}$ 。速度调整后的能耗函数可写为

$$E^{(r)}(s_S, s_D) = E_S^{0,(r)} \left[\frac{\lambda_S}{s_S} + (1 - \lambda_S) ((1 - \rho_S) + \rho_S s_S^2) \right] + E_D^{0,(r)} \left[\frac{\lambda_D}{s_D} + (1 - \lambda_D) ((1 - \rho_D) + \rho_D s_D^2) \right].$$

其中， λ_S, λ_D 用于表示低速时任务持续时间增加带来的保持能耗影响， ρ_S, ρ_D 用于表示高速时驱动能耗增加的影响。不同 seed 的能耗差异通过 $E_S^{0,(r)}$ 和 $E_D^{0,(r)}$ 体现。

为了保证同一组速度对所有 seed 都有效，需要对每一个 seed 都施加截止时间约束：

$$C_{\max}^{(r)}(s_S, s_D) \leq D, \quad \forall r \in \mathcal{R}.$$

该约束表示：无论物块随机分布如何变化，只要属于同一任务数量组合，统一速度方案都必须满足给定截止时间 D 。

本问题的目标是最小化最坏情形能耗。直接写可以表示为

$$\min_{s_S, s_D} \max_{r \in \mathcal{R}} E^{(r)}(s_S, s_D).$$

该目标函数表示：在所有 seed 中找到能耗最大的那一个，并使这个最坏能耗尽可能小。由于 $\max$ 函数直接出现在目标函数中不便于后续求解，本文引入辅助变量 z ，将最坏情形能耗目标转化为上界约束：

$$E^{(r)}(s_S, s_D) \leq z, \quad \forall r \in \mathcal{R}.$$

这样，最小化最坏情形能耗就等价于最小化 z 。

因此，问题二的鲁棒速度优化模型可写为

$$\begin{aligned} & \min_{s_S, s_D, z} \quad z \\ & \text{s.t.} \quad C_{\max}^{(r)}(s_S, s_D) \leq D, \quad \forall r \in \mathcal{R}, \\ & \quad E^{(r)}(s_S, s_D) \leq z, \quad \forall r \in \mathcal{R}, \\ & \quad s_{\min} \leq s_S \leq s_{\max}, \quad s_{\min} \leq s_D \leq s_{\max}. \end{aligned}$$

从函数性质上看， $1/s$ 在 $s > 0$ 上为凸函数，因此每个 seed 的时间函数 $C_{\max}^{(r)}(s_S, s_D)$ 是凸函数；能耗函数由 $1/s$ 项和 s^2 项加权构成，在参数非负时也具有凸性。最坏情形目

标本质上是多个凸能耗函数的最大值，而凸函数的最大值仍为凸函数。引入变量 z 后，原来的问题被转化为带一组能耗上界约束的标准形式，便于后续使用逐次线性规划法进行数值求解。

4.2.2 实现过程

鲁棒速度模式选择问题采用逐次线性规划法求解。与问题一不同，问题二不再只处理平均意义下的一组完工时间和能耗，而是同时考虑同一任务数量组合下的多个 seed。每个 seed 都会产生一组截止时间约束和能耗上界约束，因此该问题虽然决策变量仍然较少，但约束数量明显增加。

本问题的原始目标可以写成最小化所有 seed 中的最大能耗：

$$\min_{s_S, s_D} \max_{r \in \mathcal{R}} E^{(r)}(s_S, s_D).$$

其中， $\mathcal{R}$ 表示 seed 集合， $E^{(r)}(s_S, s_D)$ 表示 seed r 下的能耗函数。直接处理 $\max$ 函数会带来两个问题：一是目标函数在多个 seed 同时达到最大值时可能不可微；二是后续 KKT 验证和数值求解都不方便。因此，本文引入辅助变量 z ，将 $\min$ - $\max$ 问题改写为上界形式：

$$E^{(r)}(s_S, s_D) \leq z, \quad \forall r \in \mathcal{R}.$$

这样，最小化最坏情形能耗就等价于最小化 z 。改写后的目标函数变为线性函数：

$$\min_{s_S, s_D, z} z.$$

这种处理相当于用变量 z 表示所有 seed 能耗的共同上界。只要 z 越小，所有 seed 中的最大能耗就越小。

令

$$x = \begin{bmatrix} s_S \\ s_D \\ z \end{bmatrix}.$$

则鲁棒问题中的约束可以统一写成

$$g_q(x) \geq 0, \quad q = 1, 2, \dots, Q.$$

其中， Q 表示所有约束的总数，包括所有 seed 的截止时间约束、所有 seed 的能耗上界约束，以及速度上下界约束。具体来说，对每个 seed r ，有截止时间约束

$$g_T^{(r)}(x) = D - C_{\max}^{(r)}(s_S, s_D) \geq 0,$$

以及能耗上界约束

$$g_E^{(r)}(x) = z - E^{(r)}(s_S, s_D) \geq 0.$$

此外还有速度边界约束

$$s_S - s_{\min} \geq 0, \quad s_{\max} - s_S \geq 0,$$

$$s_D - s_{\min} \geq 0, \quad s_{\max} - s_D \geq 0.$$

从函数性质看，本问题适合使用逐次线性规划法。首先， s_S 和 s_D 均有正的下界，因此 $1/s_S$ 、 $1/s_D$ 、 s_S^2 和 s_D^2 在可行域内都是连续可导函数。其次，时间函数中含有 $1/s$ 项，能耗函数中含有 $1/s$ 和 s^2 项，这些函数在正数范围内具有凸性。因此，原问题可以看作一个具有凸结构的非线性规划问题。再次，变量只有 s_S, s_D, z 三个，但约束来自多个 seed，逐次线性规划法可以在保留多约束结构的同时，把每一步转化为小规模线性规划子问题，适合本文自编程序实现。

逐次线性规划法的核心思想是：在当前迭代点附近，用一阶 Taylor 展开近似原来的非线性约束，从而得到一个线性规划子问题。设第 k 次迭代的当前点为

$$x^{(k)} = \begin{bmatrix} s_S^{(k)} \\ s_D^{(k)} \\ z^{(k)} \end{bmatrix}.$$

对于任意一个非线性约束 $g_q(x) \geq 0$ ，在 $x^{(k)}$ 处进行一阶 Taylor 展开：

$$g_q(x) \approx g_q(x^{(k)}) + \nabla g_q(x^{(k)})^T (x - x^{(k)}).$$

于是，在第 k 次迭代中，用线性约束

$$g_q(x^{(k)}) + \nabla g_q(x^{(k)})^T (x - x^{(k)}) \geq 0$$

近似原来的非线性约束。

需要注意的是，线性化只在当前点附近可靠。如果一步走得太远，一阶近似可能与原非线性函数偏差较大，导致候选解虽然满足线性化约束，但代回原模型后不可行。因此，本文在逐次线性规划中加入步长限制，也可以理解为信赖区域约束：

$$|s_S - s_S^{(k)}| \leq \Delta_k,$$

$$|s_D - s_D^{(k)}| \leq \Delta_k.$$

其中， Δ_k 表示第 k 次迭代的信赖区域大小。该限制保证每一次线性近似只在当前点附近使用，从而提高迭代稳定性。

因此，第 k 次迭代需要求解的线性规划子问题为

$$\begin{aligned}
& \min_{s_S, s_D, z} \quad z \\
& \text{s.t.} \quad g_q(x^{(k)}) + \nabla g_q(x^{(k)})^T (x - x^{(k)}) \geq 0, \quad q = 1, 2, \dots, Q, \\
& \quad |s_S - s_S^{(k)}| \leq \Delta_k, \\
& \quad |s_D - s_D^{(k)}| \leq \Delta_k, \\
& \quad s_{\min} \leq s_S \leq s_{\max}, \\
& \quad s_{\min} \leq s_D \leq s_{\max}.
\end{aligned}$$

该子问题的目标函数是线性的，所有约束也都是线性的，因此是一个标准线性规划问题。

在线性规划子问题求解后，程序不会直接接受候选解，而是将候选解代回原来的非线性模型中检查。若候选解满足所有 seed 的原始截止时间约束：

$$C_{\max}^{(r)}(s_S, s_D) \leq D, \quad \forall r \in \mathcal{R},$$

同时满足所有 seed 的能耗上界约束：

$$E^{(r)}(s_S, s_D) \leq z, \quad \forall r \in \mathcal{R},$$

并且目标值 z 相比当前解有所改善，则接受该候选解作为新的迭代点。若候选解代回原模型后不可行，或者目标改善不明显，则说明当前线性近似不够可靠，程序缩小信赖区域 Δ_k ，重新求解局部线性规划子问题。

由于本问题的线性规划子问题只有三个变量 s_S, s_D, z ，规模较小，本文在实现中采用顶点枚举思想求解。线性规划的最优解若存在，通常可以在可行域的某个顶点取得。因此，程序将线性化后的约束整理为若干半空间约束，枚举可能的约束交点作为候选顶点，再筛选满足所有线性约束的可行点，并从中选取 z 最小的点作为线性规划子问题的解。这样可以避免依赖外部大型优化求解器，也与本文主问题中自编算法的思路保持一致。

这个过程如图 5 所示。



图 5 逐次线性规划法求解鲁棒速度模式选择问题的流程

该方法的优点在于，它没有把多个 seed 的结果简单平均，而是把每个 seed 的时间约束和能耗约束都保留下来；同时又通过逐次线性化，把复杂的非线性鲁棒优化问题转化为一系列小规模线性规划问题。最终程序输出统一速度倍率 s_S^*, s_D^* 、最坏情形能耗 z^* 、各 seed 的可行性检查结果以及 KKT 验证结果。由此可以进一步比较二臂和三臂模式在随机任务分布扰动下的稳健性差异。

4.2.3 KKT 验证条件

鲁棒速度模式选择问题的 KKT 条件比平均速度-能耗优化问题更加复杂。其标准形式为

$$\begin{aligned}
 \min_x \quad & z \\
 \text{s.t.} \quad & g_q(x) \geq 0, \quad q = 1, 2, \dots, Q.
 \end{aligned}$$

其中， $x = (s_S, s_D, z)^T$ ， Q 包含所有 seed 的能耗上界约束、截止时间约束以及速度上下界约束。

其 KKT 条件可以写为

$$\begin{aligned}\nabla z - \sum_{q=1}^Q \mu_q \nabla g_q(x^*) &= 0, \\ g_q(x^*) &\geq 0, \quad q = 1, 2, \dots, Q, \\ \mu_q &\geq 0, \quad q = 1, 2, \dots, Q, \\ \mu_q g_q(x^*) &= 0, \quad q = 1, 2, \dots, Q.\end{aligned}$$

其中，平稳性条件要求目标变量 z 与各个 seed 的能耗约束、时间约束以及速度边界约束之间形成一阶平衡关系；互补松弛条件用于判断哪些 seed 或边界约束在最优点附近真正起作用。

需要注意的是，鲁棒问题采用逐次线性规划法求解，最终结果首先保证原非线性约束下的鲁棒可行性；KKT 检查则进一步用于判断该数值解是否能够作为严格的一阶局部最优点。若部分行未通过 KKT 检查，则应将其解释为鲁棒可行解或数值近似解，而不能直接称为严格最优解。

5 数据分析与结果讨论

5.1 具体数据

5.1.1 主问题数据

主问题数据主要来自附件一 outputs 中 four case framework 和 mode decision 部分，并将其输出结果整合为了附件二中的 01–13 号工作表，并在附件三中整理了单独运行调试结果。其中，01–08 号工作表对应 CP-SAT 求解器、融合剪枝的隐枚举法、启发式算法和热启动后的隐枚举法在二臂和三臂场景下的单独调度结果；09–12 号工作表对应四种方法的二臂/三臂时间能耗对比结果；13 号工作表对应二臂/三臂模式决策汇总。

在实验设置上，本文对 16 种任务数量组合和 3 个随机 seed 进行批量实验。每一种任务数量组合用 counts_code 表示，例如 1111 表示 Type1、Type2、Type3、Type4 四类任务各有 1 个，2222 表示四类任务各有 2 个。不同 seed 表示同一任务规模下不同的物块随机空间分布，因此能够反映任务位置变化对调度结果的影响。

二臂/三臂模式决策的具体结果如表 ?? 所示。表中每一行对应一种任务数量组合，数据为同一 counts_code 下 3 个随机 seed 的平均结果。“ latex 四种方法在 16 种任务组合下的平均结果分别如表 5–表 8 所示。每一行对应一种 Type 任务数量组合，表中数值为该组合下 3 个随机 seed 的平均值。

表 5 CP-SAT 求解器在 16 种任务组合下的平均结果

Type1	Type2	Type3	Type4	$C_{\max}^{2B}$	E_{tot}^{2B}	$C_{\max}^{3B}$	E_{tot}^{3B}	平均计算时间/s
1	1	1	1	1864.00	8015.00	1144.33	8445.00	1.1087
1	1	1	2	2390.33	12076.33	1674.67	12578.00	0.4266
1	1	2	1	2341.00	9844.00	1646.67	10282.33	0.3741
1	1	2	2	2864.67	13975.67	2172.67	14493.33	2.8354
1	2	1	1	2160.33	8401.00	1387.00	8807.67	0.3500
1	2	1	2	2689.33	12514.67	1667.33	12997.00	0.9333
1	2	2	1	2799.67	11139.67	1842.00	11610.33	1.6254
1	2	2	2	3300.67	15014.33	2285.00	15531.33	17.3515
2	1	1	1	1955.33	7985.67	1303.33	8454.00	0.3118
2	1	1	2	2433.00	11829.33	1644.33	12374.00	0.9109
2	1	2	1	2573.00	10464.67	1746.00	10947.67	1.0879
2	1	2	2	3082.00	14612.00	2295.33	15207.00	19.1113
2	2	1	1	2077.67	8824.67	1505.33	9260.33	0.6799
2	2	1	2	2638.33	13213.00	1794.00	13787.33	3.2290
2	2	2	1	2572.33	10689.67	1749.00	11194.67	3.6314
2	2	2	2	3125.00	15068.33	2282.00	15655.00	62.2421

表 6 融合剪枝的隐枚举法在 16 种任务组合下的平均结果

Type1	Type2	Type3	Type4	$C_{\max}^{2B}$	E_{tot}^{2B}	$C_{\max}^{3B}$	E_{tot}^{3B}	平均计算时间/s
1	1	1	1	1864.00	8015.00	1144.33	8445.00	0.0160
1	1	1	2	2390.33	12076.33	1674.67	12578.00	0.1280
1	1	2	1	2341.00	9844.00	1646.67	10282.33	0.1290
1	1	2	2	2864.67	13975.67	2172.67	14493.33	1.7181
1	2	1	1	2160.33	8401.00	1387.00	8807.67	0.1129
1	2	1	2	2689.33	12514.67	1667.33	12997.00	0.5551
1	2	2	1	2799.67	11139.67	1842.00	11610.33	0.7532
1	2	2	2	3300.67	15014.33	2285.00	15531.33	18.3875
2	1	1	1	1955.33	7985.67	1303.33	8454.00	0.1284
2	1	1	2	2433.00	11829.33	1644.33	12374.00	0.7779
2	1	2	1	2573.00	10464.67	1746.00	10947.67	0.8380
2	1	2	2	3082.00	14612.00	2295.33	15207.00	27.9397
2	2	1	1	2077.67	8824.67	1505.33	9260.33	1.2688
2	2	1	2	2638.33	13213.00	1794.00	13787.33	3.3501
2	2	2	1	2572.33	10689.67	1749.00	11194.67	4.2431
2	2	2	2	3125.00	15068.33	2282.00	15655.00	94.0316

表 7 启发式算法在 16 种任务组合下的平均结果

Type1	Type2	Type3	Type4	$C_{\max}^{2B}$	E_{tot}^{2B}	$C_{\max}^{3B}$	E_{tot}^{3B}	平均计算时间/s
1	1	1	1	1864.00	8015.00	1144.33	8445.00	0.1873
1	1	1	2	2390.33	12076.33	1674.67	12578.00	0.2025
1	1	2	1	2341.00	9844.00	1646.67	10289.00	0.2024
1	1	2	2	2864.67	13975.67	2172.67	14493.33	1.0874
1	2	1	1	2160.33	8401.00	1398.00	8833.67	0.2134
1	2	1	2	2689.33	12514.67	1676.33	13005.33	1.0964
1	2	2	1	2804.00	11122.33	1842.00	11610.33	1.1362
1	2	2	2	3305.00	14997.00	2286.00	15533.33	1.0779
2	1	1	1	1955.33	7985.67	1338.33	8482.00	0.2060
2	1	1	2	2433.00	11829.33	1654.33	12395.33	1.0458
2	1	2	1	2573.00	10464.67	1749.00	10945.00	1.1137
2	1	2	2	3082.00	14612.00	2298.00	15211.33	1.2413
2	2	1	1	2079.33	8815.67	1535.67	9255.67	1.1127
2	2	1	2	2638.33	13213.00	1967.67	13797.33	1.2333
2	2	2	1	2572.33	10689.67	1840.00	11189.33	1.0568
2	2	2	2	3131.67	15072.67	2282.67	15678.67	1.8600

表 8 热启动后的隐枚举法在 16 种任务组合下的平均结果

Type1	Type2	Type3	Type4	$C_{\max}^{2B}$	E_{tot}^{2B}	$C_{\max}^{3B}$	E_{tot}^{3B}	平均计算时间/s
1	1	1	1	1864.00	8015.00	1144.33	8445.00	0.0374
1	1	1	2	2390.33	12076.33	1674.67	12578.00	0.1635
1	1	2	1	2341.00	9844.00	1646.67	10282.33	0.1564
1	1	2	2	2864.67	13975.67	2172.67	14493.33	1.6760
1	2	1	1	2160.33	8401.00	1387.00	8807.67	0.1565
1	2	1	2	2689.33	12514.67	1667.33	12997.00	0.6132
1	2	2	1	2799.67	11139.67	1842.00	11610.33	0.7846
1	2	2	2	3300.67	15014.33	2285.00	15531.33	9.8419
2	1	1	1	1955.33	7985.67	1303.33	8454.00	0.1477
2	1	1	2	2433.00	11829.33	1644.33	12374.00	0.7320
2	1	2	1	2573.00	10464.67	1746.00	10947.67	0.8028
2	1	2	2	3082.00	14612.00	2295.33	15207.00	23.5496
2	2	1	1	2077.67	8824.67	1505.33	9260.33	0.5756
2	2	1	2	2638.33	13213.00	1794.00	13787.33	4.1532
2	2	2	1	2572.33	10689.67	1749.00	11194.67	4.7159
2	2	2	2	3125.00	15068.33	2282.00	15655.00	48.9079

5.1.2 延伸问题数据

主问题数据主要来自附件一 outputs 中 nonlinear programming 部分，并将其输出结果整合为了附件二中的 14-17 号工作表，问题一为速度-能耗非线性优化问题，数据来自 15 调速能耗优化和 14 调速 KKT 验证；问题二为鲁棒速度模式选择问题，数据来自 17 鲁棒调速模式选择”和 16 鲁棒 KKT 验证。

由于原始 Excel 中问题一输出结果共有 1024 行，问题二输出结果共有 256 行，若全部列入正文会造成篇幅过长。因此，本文采用“整体统计+代表性抽样”的方式展示数

据：整体统计用于说明总体趋势，代表性抽样用于展示具体任务组合、截止时间比例、速度倍率、能耗和推荐模式之间的关系。

问题一：速度-能耗非线性优化输出结果 问题一对同一任务数量组合下多个 seed 的主问题结果取平均，并在平均结果基础上优化单臂任务速度倍率和双臂协同任务速度倍率。该问题共考虑 16 种任务数量组合、16 个截止时间比例和 4 组速度-能耗参数，因此主结果共有

$$16 \times 16 \times 4 = 1024$$

行。其整体输出结果如表 9 所示。

表 9 问题一速度-能耗非线性优化整体输出统计

数据范围	主结果行数	推荐二臂	推荐三臂	三臂比例	二臂平均能耗	三臂平均能耗
全部参数组	1024	795	229	22.36%	11675.21	11827.55
基准参数组	256	209	47	18.36%	11690.09	11948.25

为了进一步展示具体计算结果，表 10 选取了若干具有代表性的任务组合和截止时间比例。表中 s_S 表示单臂任务速度倍率， s_D 表示双臂协同任务速度倍率， E^{2B*} 和 E^{3B*} 分别表示二臂、三臂调速后的优化能耗。

表 10 问题一速度-能耗非线性优化代表性输出样例

参数组	任务组合	截止比例	截止时间	二臂速度 (s_S, s_D)	E^{2B*}	$C_{\max}^{2B*}$	三臂速度 (s_S, s_D)	E^{3B*}	$C_{\max}^{3B*}$	推荐模式
基准参数	1221	1.00	2799.67	(0.9795, 1.0214)	11136.16	2799.67	(0.9163, 0.9687)	11579.97	1955.90	二臂
低慢速惩罚	2122	0.70	2157.40	(1.4413, 1.4192)	17517.08	2157.40	(1.0556, 1.0703)	15486.80	2157.40	三臂
低慢速惩罚	2122	0.80	2465.60	(1.2539, 1.2471)	16012.73	2465.60	(0.9107, 0.9468)	14982.40	2465.60	三臂
低慢速惩罚	2221	0.70	1800.63	(1.4380, 1.4162)	12813.07	1800.63	(0.9594, 0.9877)	11115.36	1800.63	三臂
基准参数	2222	0.70	2187.50	(1.4314, 1.4257)	16984.85	2187.50	(1.0259, 1.0611)	15720.41	2187.50	三臂

由表 10 可以看出，推荐模式会随任务组合、截止时间比例和速度-能耗参数发生变化。例如，在 1221 且截止比例为 1.00 的基准参数组下，二臂优化后能耗低于三臂，因此推荐二臂；而在 2122、2221 和 2222 的紧工期样例中，三臂虽然需要承担额外系统能耗，但由于并行能力更强、所需速度倍率较低，调速后能耗反而低于二臂，因此推荐三臂。这说明问题一的结果不是简单地固定推荐某一种模式，而是由任务结构、工期约束和速度-能耗曲线共同决定。

问题一：速度-能耗非线性优化 KKT 验证结果 问题一的 KKT 验证结果来自“16 调速 KKT 验证”工作表。由于问题一共有 1024 行主结果，且每一行分别检查二臂和三臂两个模式，因此 KKT 验证结果共有

$$1024 \times 2 = 2048$$

行。其整体统计结果如表 11 所示。

表 11 问题一速度-能耗非线性优化 KKT 整体验证结果

数据范围	KKT 检查行数	通过行数	未通过行数	通过率
全部参数组	2048	2045	3	99.85%
基准参数组	512	512	0	100.00%

为了说明 KKT 验证并不是只给出最终通过率，表 12 给出若干典型检查行。表中“活跃约束”用于说明最优点附近主要由哪类约束起作用。

表 12 问题一速度-能耗非线性优化 KKT 验证样例

任务组合	截止比例	模式	最优速度 (s_S, s_D)	优化能耗	优化后完工时间	活跃约束	KKT 是否通过
1122	0.80	二臂	(1.2436, 1.2532)	14704.08	2291.73	截止时间约束	是
1122	0.80	三臂	(0.9163, 0.9687)	14464.63	2285.60	无	是
2111	0.75	二臂	(1.3325, 1.3346)	8669.77	1466.50	截止时间约束	是
2111	0.75	三臂	(0.9163, 0.9687)	8428.68	1391.63	无	是

由表 11 和表 12 可知，问题一的数值解整体稳定。数结果不仅满足速度上下界和截止时间约束，而且能够通过 KKT 一阶最优性检查。。

问题二：鲁棒速度模式选择输出结果 问题二不再对多个 seed 取平均，而是要求同一任务数量组合下所有 seed 共用同一组速度倍率，并最小化最坏情形能耗。该问题共考虑 16 种任务数量组合和 16 个截止时间比例，因此主结果共有

$$16 \times 16 = 256$$

行。其整体输出结果如表 13 所示。

表 13 问题二鲁棒速度模式选择整体输出统计

模式	鲁棒可行组数	推荐次数	平均最坏能耗	单臂速度倍率	双臂速度倍率
二臂模式	256	187	12404.60	1.0768	1.1196
三臂模式	256	69	12500.64	0.9247	0.9766

为了展示鲁棒问题中推荐模式如何随截止时间变化，表 14 选取了 1111、1122 和 2222 三组任务组合的代表性结果。表中 Z^{2B} 和 Z^{3B} 表示二臂、三臂模式下的最坏情形能耗上界。

表 14 问题二鲁棒速度模式选择代表性输出样例

任务组合	截止比例	截止时间	二臂速度 (s_S, s_D)	Z^{2B}	二臂最大完工时间	三臂速度 (s_S, s_D)	Z^{3B}	三臂最大完工时间	推荐模式
1111	0.75	1398.00	(1.4031, 1.4254)	9658.23	1398.00	(0.9163, 0.9687)	9055.26	1333.67	三臂
1111	0.80	1491.20	(1.3115, 1.3403)	9324.95	1491.20	(0.9163, 0.9687)	9055.26	1333.67	三臂
1111	0.85	1584.40	(1.2313, 1.2648)	9076.26	1584.40	(0.9163, 0.9687)	9055.26	1333.67	三臂
1122	0.85	2434.97	(1.2173, 1.2514)	14983.77	2434.97	(0.9163, 0.9687)	14785.66	2300.69	三臂
1122	0.90	2578.20	(1.1454, 1.1842)	14694.07	2578.20	(0.9163, 0.9687)	14785.66	2300.69	二臂
1122	0.95	2721.43	(1.0814, 1.1239)	14491.85	2721.43	(0.9163, 0.9687)	14785.66	2300.69	二臂
2222	0.70	2187.50	(1.4835, 1.5017)	17747.13	2187.50	(1.0603, 1.1038)	16051.60	2187.50	三臂
2222	0.75	2343.75	(1.3809, 1.4055)	16996.60	2343.75	(0.9859, 1.0342)	15899.76	2343.75	三臂
2222	0.80	2500.00	(1.2912, 1.3211)	16435.10	2500.00	(0.9211, 0.9731)	15850.69	2500.00	三臂

由表 14 可以看出，鲁棒问题的推荐结果与截止时间比例密切相关。在 1122 组合中，当截止比例为 0.85 时，三臂最坏能耗低于二臂，因此推荐三臂；当截止比例放宽到

0.90 和 0.95 时，二臂最坏能耗反而更低，因此推荐二臂。这说明鲁棒问题并不是简单地因为三臂资源更多就总是推荐三臂，而是在紧工期下三臂更有优势，在工期逐渐宽松后二臂的低能耗优势会重新体现出来。

问题二：鲁棒速度模式选择 KKT 验证结果 由于问题二共有 256 行主结果，且每一行分别检查二臂和三臂两个模式，因此 KKT 验证结果共有

$$256 \times 2 = 512$$

行。其整体统计结果如表 15 所示。

表 15 问题二鲁棒速度模式选择 KKT 整体验证结果

检查对象	KKT 检查行数	通过行数	未通过行数	通过率
二臂模式	256	107	149	41.80%
三臂模式	256	224	32	87.50%
合计	512	331	181	64.65%

为了说明鲁棒 KKT 验证并非全部失败，表 16 选取了若干代表性检查行，既包含 KKT 未通过的紧工期结果，也包含 KKT 通过的结果。表中可以看出，所有样例均满足鲁棒可行性，但是否满足严格 KKT 检查与活跃约束结构和平稳性残差有关。

表 16 问题二鲁棒速度模式选择 KKT 验证样例

任务组合	截止比例	模式	最优速度 (s_S, s_D)	最坏能耗 z	调速后最大完工时间	活跃约束	平稳性残差范数	KKT 是否通过
1111	0.70	二臂	(1.5070, 1.5234)	10101.39	1304.80	时间种子 1 约束 + 能耗种子 2 约束	0.999967	否
1111	0.70	三臂	(0.9378, 0.9887)	9058.01	1304.80	时间种子 2 约束 + 能耗种子 2 约束	0.995768	否
1111	0.75	三臂	(0.9163, 0.9687)	9055.26	1333.67	能耗种子 2 约束	0.000000	是
1111	1.15	二臂	(0.9163, 0.9687)	8597.46	2099.25	能耗种子 2 约束	0.000000	是
1122	0.80	三臂	(0.9203, 0.9723)	14785.81	2291.70	时间种子 2 约束 + 能耗种子 2 约束	0.810700	否
1122	0.85	三臂	(0.9163, 0.9687)	14785.66	2300.69	能耗种子 2 约束	0.000000	是
2211	0.85	三臂	(0.9163, 0.9687)	10076.15	1739.35	能耗种子 0 约束	0.000000	是
2222	0.80	三臂	(0.9211, 0.9731)	15850.69	2500.00	时间种子 0 约束 + 能耗种子 0 约束	0.947640	否
2222	0.85	三臂	(0.9163, 0.9687)	15850.45	2512.31	能耗种子 0 约束	0.000000	是

由表 15 和表 16 可以看出，鲁棒问题的 KKT 通过率明显低于问题一。这并不说明鲁棒结果没有意义，而是说明逐次线性规划法得到的结果更适合解释为鲁棒可行解或数值近似解。由于鲁棒模型同时包含多个 seed 的时间约束和能耗上界约束，活跃约束集合可能在不同 seed 之间切换，乘子恢复和平稳性检查更敏感，因此 KKT 通过率低于问题一是合理的。

5.2 解的评估与结果解释

对于主问题，由于 CP-SAT 求解器是成熟的标准求解器，因此本文将 CP-SAT 结果作为对照组，比较自编算法与其在最大完工时间、系统总能耗和求解时间上的差异。若自编方法得到的目标值与 CP-SAT 保持一致，则说明该方法在对应算例上达到了标准求解器结果；若存在差异，则进一步分析差异是否来自启发式近似或目标优先级差异。

对于非线性延伸问题，通过 KKT 条件验证数值解的可靠性。问题一为速度-能耗非线性优化问题，目标函数和约束形式较规整，KKT 验证通过率较高；问题二为鲁棒速度模式选择问题，涉及多个 seed 的最坏情形约束，采用逐次线性规划近似求解，因此 KKT 通过率较低，需要区分鲁棒可行解和严格 KKT 最优解。

5.2.1 主问题解的评估

主问题的解主要从四种求解方法之间的对照关系进行评价。本文将 CP-SAT 求解器作为标准对照组，因为 CP-SAT 能够直接处理 0-1 变量、可选区间变量、资源互斥约束和多阶段目标，在本文主问题中可作为判断其他方法结果是否合理的基准。

从上述数据可以看出，CP-SAT 求解器、融合剪枝的隐枚举法和热启动后的隐枚举法在二臂平均完工时间、二臂平均能耗、三臂平均完工时间和三臂平均能耗上完全一致。具体来说，三种方法均得到二臂平均完工时间 2554.17、二臂平均能耗 11479.25、三臂平均完工时间 1758.69 和三臂平均能耗 11976.56。这说明融合剪枝的隐枚举法和热启动后的隐枚举法在这些算例上达到了与 CP-SAT 相同的目标值，其求解结果是合理的。

该一致性也说明，本文自编精确搜索方法中的剪枝并没有错误删除最优分支。若剪枝规则过强，可能出现自编方法的结果优于或劣于 CP-SAT 的异常情况；而现在三种方法的完工时间和能耗完全一致，说明完工时间下界、能耗下界、负载下界和中心安全区互斥剪枝能够减少搜索量，同时保留最优解。因此，融合剪枝的隐枚举法不仅在逻辑上符合分支定界思想，在数值结果上也得到了标准求解器的验证。

启发式算法的结果与 CP-SAT 并不完全一致，但这种差异是合理的。启发式算法的二臂平均完工时间为 2555.23，略高于 CP-SAT 的 2554.17；三臂平均完工时间为 1781.65，高于 CP-SAT 的 1758.69。这说明启发式算法没有在所有算例上达到全局最优，尤其在三臂场景中偏差更明显。

这种偏差并不表示启发式结果无效，而是由启发式方法的性质决定的。启发式算法主要依靠规则构造和局部改进，目标是在较短时间内生成质量较好的可行解，而不是保证遍历完整搜索空间。三臂系统中双臂协同任务的可选组合更多，任务顺序和机械臂组合之间的耦合更强，因此启发式规则更容易陷入局部较优方案。表 ?? 中二臂模式有 44/48 组达到最优，而三臂模式只有 23/48 组达到最优，也说明三臂场景的搜索难度更高。

5.2.2 非线性延伸问题解的评估

非线性延伸问题的解主要通过 KKT 条件进行评价。KKT 条件包括原问题可行性、乘子非负性、互补松弛和平稳性条件。对于本文的凸优化问题，在满足一定正则性条件的情况下，KKT 条件不仅是局部最优的一阶必要条件，也可以作为判断数值解是否具有最优性的重要依据。因此，本文在非线性的部分不只检查解是否满足约束，还进一步检查其是否满足 KKT 条件。

对于问题一，即速度-能耗非线性优化问题，KKT 验证结果整体较好。由表 11 可知，全部参数组下共有 2048 行 KKT 检查结果，其中 2045 行通过，未通过仅 3 行，通过率达到 99.85%；基准参数组下 512 行全部通过，通过率为 100.00%。

该结果说明，内点制约函数法结合 0.618 一维搜索得到的速度解，在绝大多数情况下不仅满足速度上下界和截止时间约束，也满足互补松弛和平稳性检查。由于问题一只有两个主要连续变量 s_S 和 s_D ，且时间函数由 $1/s$ 项构成，能耗函数由 $1/s$ 和 s^2 项构成，在正速度区间内具有较好的凸性结构，因此 KKT 检查通过率高是合理的。

对于问题一中少数未通过 KKT 的结果，不应理解为模型错误，而更可能来自数值

迭代精度、边界附近残差或容差设置。因为其未通过数量仅占全部结果的极小比例，并且总体推荐趋势和能耗比较不依赖这几行异常结果，所以问题一的数值解可以认为整体稳定，能够支持后续关于二臂/三臂速度-能耗权衡的分析。

对于问题二，即鲁棒速度模式选择问题，KKT 验证结果明显复杂。由表 15 可知，鲁棒问题共有 512 行 KKT 检查结果，其中通过 331 行，未通过 181 行，总通过率为 64.65%。其中，二臂模式通过率为 41.80%，三臂模式通过率为 87.50%。

问题二 KKT 通过率较低是可以解释的。

1. 问题二不是对平均结果进行优化，而是要求同一组速度同时满足多个 seed 的截止时间约束和能耗上界约束。也就是说，模型中存在多组时间约束和多组能耗约束，最优点可能由某一个或多个最不利 seed 决定。当最坏能耗 seed 或活跃时间约束在迭代过程中发生切换时，活跃约束集合会变得敏感，从而导致 KKT 平稳性残差较大。
2. 采用逐次线性规划法求解。该方法每一步求解的是当前点附近的线性化子问题，而不是一次性直接求解原始非线性模型。虽然程序会将候选解代回原非线性约束中检查可行性，但最终结果仍可能只是逐次线性化意义下的近似收敛点。因此，某些结果能够满足鲁棒可行性，却未必满足原始非线性问题的严格 KKT 条件。
3. 二臂模式 KKT 通过率低于三臂模式也是合理的。二臂系统资源较少，为了让所有 seed 都满足同一截止时间，往往需要更高速度，并且更容易使时间约束处于活跃状态。当时间约束和能耗上界约束同时接近边界时，乘子恢复和平稳性检查难度会增加，因此二臂模式更容易出现 KKT 未通过。三臂系统由于并行能力更强，时间压力相对较小，活跃约束结构更稳定，所以 KKT 通过率更高。

因此，对问题二结果的解释应更加谨慎。KKT 未通过并不表示该行结果不可行，也不表示鲁棒模型没有意义；它只说明该结果不能被严格称为满足一阶最优性的局部最优解。对于 KKT 未通过的结果，应表述为“鲁棒可行解”或“数值近似解”；对于 KKT 通过的结果，则可以进一步认为其满足一阶最优性检查。

从工程分析角度看，问题二仍然有意义。因为鲁棒模型的首要目标是找到一组统一速度，使同一任务数量组合下所有 seed 都满足截止时间约束，并尽量控制最坏情形能耗。即使部分结果未通过严格 KKT 检查，只要它们通过原非线性约束的可行性检查，仍然可以用于比较二臂和三臂在 seed 扰动下的稳健性差异。报告中应避免把这些结果表述为严格最优解，而应表述为鲁棒可行的数值近似结果。

综上，问题一的 KKT 通过率接近 100%，说明内点法与 0.618 搜索得到的速度-能耗优化结果整体可靠；问题二由于采用逐次线性规划法，并且包含多个 seed 的最坏情形约束，KKT 通过率较低，因此其结果更适合解释为鲁棒可行解和数值近似解，而非严格最优解。

5.3 方法对比

5.3.1 主问题四种方法对比

四种方法在 16 种任务组合下的最优性与平均计算时间如表 17 所示。其中，计算时间为同一任务组合下不同 seed 和不同机械臂模式的平均求解时间。

表 17 主问题四种方法的最优性与计算时间对比

任务组合	方法 1 是否最优	方法 1 时间/s	方法 2 是否最优	方法 2 时间/s	方法 3 是否最优	方法 3 时间/s	方法 4 是否最优	方法 4 时间/s
1111	是	1.1087	是	0.0160	是	0.1873	是	0.0374
1112	是	0.4266	是	0.1280	是	0.2025	是	0.1635
1121	是	0.3741	是	0.1290	否	0.2024	是	0.1564
1122	是	2.8354	是	1.7181	是	1.0874	是	1.6760
1211	是	0.3500	是	0.1129	否	0.2134	是	0.1565
1212	是	0.9333	是	0.5551	否	1.0964	是	0.6132
1221	是	1.6254	是	0.7532	否	1.1362	是	0.7846
1222	是	17.3515	是	18.3875	否	1.0779	是	9.8419
2111	是	0.3118	是	0.1284	否	0.2060	是	0.1477
2112	是	0.9109	是	0.7779	否	1.0458	是	0.7320
2121	是	1.0879	是	0.8380	否	1.1137	是	0.8028
2122	是	19.1113	是	27.9397	否	1.2413	是	23.5496
2211	是	0.6799	是	1.2688	否	1.1127	是	0.5756
2212	是	3.2290	是	3.3501	否	1.2333	是	4.1532
2221	是	3.6314	是	4.2431	否	1.0568	是	4.7159
2222	是	62.2421	是	94.0316	否	1.8600	是	48.9079

由表 17 可以看出，方法 1、方法 2 和方法 4 在 16 种任务组合下均能够得到与 CP-SAT 一致的结果，因此这三种方法都可以认为能够找到主问题的最优解。方法 3 的计算时间最短，平均仅为 0.8796 s，但其结果并不总是与 CP-SAT 一致，因此启发式算法更适合作为快速可行解生成方法，而不适合作为严格最优解算法单独使用。

在能够找到最优解的三种方法中，方法 2 与方法 1 的速度关系随任务规模变化而变化。当任务数量较少、组合结构较简单时，方法 2 往往比方法 1 更快。例如在 1111、1112、1121、1211 等组合中，方法 2 的计算时间均低于方法 1。这说明在小规模算例中，自编隐枚举法结合剪枝可以较快排除无效分支，搜索效率较高。

但是，当任务数量增加，尤其是 Type3、Type4 等双臂协同任务数量增加时，方法 2 的搜索空间迅速扩大，计算时间开始超过方法 1。例如在 1222 组合中，方法 2 用时 18.3875 s，高于方法 1 的 17.3515 s；在 2122 组合中，方法 2 用时 27.9397 s，高于方法 1 的 19.1113 s；在 2222 组合中，方法 2 用时达到 94.0316 s，也高于方法 1 的 62.2421 s。这说明单纯依靠剪枝的隐枚举法虽然能够保证最优性，但在较大规模组合下仍会受到搜索空间膨胀的影响。

方法 4 在保持最优性的同时，整体计算时间最优。其平均计算时间为 6.0634 s，低于方法 1 的 7.2631 s 和方法 2 的 9.6486 s。尤其在较大规模组合下，方法 4 的优势更加明显。例如 1222 组合中，方法 4 用时 9.8419 s，明显低于方法 1 和方法 2；2222 组合中，方法 4 用时 48.9079 s，也低于方法 1 的 62.2421 s 和方法 2 的 94.0316 s。

这说明热启动策略是有效的。方法 4 先利用启发式算法快速得到一个较好的初始可行解，再以该解作为隐枚举搜索的初始上界，从而使大量不可能优于当前解的分支被提前剪掉。与方法 2 相比，方法 4 并没有改变隐枚举法的最优性判断逻辑，因此仍然能够得到最优解；但由于初始上界更紧，搜索效率明显提高。

综上，在四种方法中，方法 3 的速度最快但不能保证最优性；方法 1 可以作为可靠对照；方法 2 能够找到最优解，但在任务数量较多时速度下降明显；方法 4 在保持最优

性的同时具有更好的总体计算效率。因此，本文将方法 4，即热启动后的隐枚举法，作为主问题中综合表现较好的算法。

5.3.2 延伸问题方法对比

表 18 对课内常见方法与本文方法的关系进行对比。

表 18 延伸问题非线性规划方法选择对比

方法类别	课内方法特点	与本文问题的关系	本文处理方式
解析法与 KKT 条件	通过一阶条件、二阶条件或 KKT 条件描述最优应满足的数学性质	本文两个问题均含不等式约束，KKT 条件适合于检验数值解是否满足可行性、互补松弛和平稳性	用于解的验证，而不是作为主求解算法
最速下降法	根据负梯度方向迭代，计算简单，适合说明下降迭代思想	问题一、问题二都存在速度边界和截止时间约束，单纯沿负梯度方向迭代还需要额外处理可行性	吸收其“下降方向”思想，但不单独采用
牛顿法、拟牛顿法、共轭梯度法	适合无约束或经过转化后的光滑优化问题，强调利用梯度或近似 Hessian 信息提高收敛效率	本文变量维数较低，且约束条件比变量维数更重要，重点不是高维无约束搜索，而是保持约束可行并处理截止时间限制	不作为主要算法，主要保留梯度与一阶条件思想
0.618 法	用于一维搜索，在单峰区间内不断缩小最优步长所在范围	问题一在确定下降方向后，需要沿该方向选择合适步长；由于变量少、目标函数光滑，一维搜索结构清楚	与内点制约函数法结合，用于问题一的步长搜索
制约函数法	将有约束问题转化为带制约项的无约束或近似无约束问题，使迭代点保持在可行域内部	问题一具有明确的不等式约束，包括速度上下界和截止时间约束，适合用制约函数统一处理	作为问题一的主要求解方法
逐次逼近法	将复杂非线性问题在当前点附近近似为较易求解的子问题，再逐步迭代逼近原问题解	问题二包含多个 seed 的时间约束和能耗上界约束，引入辅助变量后可在当前点附近线性化为小规模线性规划子问题	采用逐次线性规划法求解问题二

对于问题一，本文选择内点制约函数法结合 0.618 法。问题一的变量只有单臂速度倍率 s_S 和双臂协同速度倍率 s_D ，约束主要是速度上下界和截止时间约束，均属于不等式约束：

$$s_{\min} \leq s_S \leq s_{\max}, \quad s_{\min} \leq s_D \leq s_{\max},$$

$$C_{\max}(s_S, s_D) \leq D.$$

因此，使用制约函数法可以将约束统一写入目标函数，使迭代过程保持在可行域内部。随后，在每一步搜索中，二维问题可以沿下降方向转化为关于步长 α 的一维搜索问题：

$$\varphi(\alpha) = \Phi_{\mu}(\mathbf{s}^{(k)} + \alpha \mathbf{d}^{(k)}).$$

此时采用 0.618 法只需要比较函数值，不需要计算二阶导数，适合本文变量少、函数形式明确的速度-能耗优化问题。

对于问题二，本文选择逐次线性规划法。问题二需要所有 seed 共用同一组速度倍率，并最小化最坏情形能耗。引入辅助变量 z 后，模型可写为

$$\min_{s_S, s_D, z} z,$$

$$E^{(r)}(s_S, s_D) \leq z, \quad C_{\max}^{(r)}(s_S, s_D) \leq D, \quad \forall r \in \mathcal{R}.$$

该问题变量数量仍然较少，但约束数量随 seed 增加而增加。逐次线性规划法可以在当前点附近对非线性约束进行一阶线性化，将原问题逐步转化为小规模线性规划子问题，便于自编程序实现，也适合处理多 seed 鲁棒约束。

综上，本文的问题一制约函数法处理约束、0.618 法确定步长，问题二采用逐次线性规划法处理多 seed 鲁棒约束。KKT 条件则用于求解后的最优性验证。这样的安排既

对应了课内非线性规划中的制约函数法、步长搜索、逐次逼近法和 KKT 条件，也与本文两个非线性问题的结构相匹配。

6 结论

6.1 研究结论

本文围绕多机械臂搬运调度与模式选择问题展开研究。该问题不是机械臂运动学或轨迹控制问题，而是将机械臂视为可分配、可协同、可调度的作业资源，从运筹学角度研究任务分配、执行顺序、资源互斥、协同搬运和模式选择问题。

在主问题建模方面，本文将多类别物块搬运任务划分为单臂搬运任务和双臂协同搬运任务，并建立了包含机械臂资源互斥、中心安全区占用、任务顺序衔接和序贯目标规划的调度模型。模型首先最小化最大完工时间，其次在不破坏时间目标的基础上最小化系统总能耗，最后考虑机械臂负载均衡，从而体现了实际搬运系统中“先保证工期，再考虑能耗和均衡”的决策逻辑。

在主问题求解方面，本文采用 CP-SAT 求解器、融合剪枝的隐枚举法、启发式算法和热启动后的隐枚举法进行对比。结果表明，CP-SAT 求解器、融合剪枝的隐枚举法和热启动后的隐枚举法在 16 种任务组合下均能够得到一致的最优结果，说明自编精确搜索方法的求解结果是可靠的。启发式算法虽然计算速度最快，但在部分算例中不能完全达到最优解，因此更适合作为快速可行解生成方法或热启动初始解。

从计算效率看，融合剪枝的隐枚举法在任务数量较少时速度较快，但随着任务数量增加，搜索空间迅速扩大，计算时间明显上升。热启动后的隐枚举法利用启发式解提供较好的初始上界，在保持最优性的同时提高了剪枝效率，整体平均计算时间低于 CP-SAT 求解器和普通隐枚举法。因此，本文认为热启动后的隐枚举法是主问题中综合表现较好的算法。

在二臂与三臂模式选择方面，计算结果表明，三臂系统能够明显缩短最大完工时间，平均时间节省率约为 31.24%，但同时平均能耗增加约为 4.46%。这说明增加第三只机械臂的主要价值在于提高并行能力和缩短工期，而不是直接降低能耗。因此，三臂系统并不是无条件优于二臂系统，其推荐与否取决于决策者对时间收益和能耗代价的权衡。

在非线性延伸问题中，本文进一步引入速度倍率变量，将离散调度结果扩展到连续速度优化。问题一建立了速度-能耗非线性优化模型，并采用内点制约函数法结合 0.618 法求解。KKT 验证结果表明，该问题在全部参数组下通过率达到 99.85%，说明所得速度解整体满足一阶最优性条件，数值结果较为可靠。

问题二进一步考虑不同 seed 下任务空间分布变化，建立鲁棒速度模式选择模型，并采用逐次线性规划法求解。其 KKT 总通过率为 64.65%，低于问题一。这说明鲁棒问题由于包含多个 seed 的时间约束和最坏能耗约束，活跃约束结构更加复杂，因此部分结果更适合解释为鲁棒可行解或数值近似解，而不能全部声称为严格最优解。但从工程意义上看，鲁棒模型能够反映不同任务位置扰动下二臂和三臂系统的稳健性差异，因此仍具有分析价值。

综上，本文的核心结论是：多机械臂搬运系统的优化不能简单理解为“机械臂越多越好”或“速度越快越好”。二臂系统在能耗控制方面具有优势，三臂系统在缩短工期

和提高鲁棒性方面具有优势。实际系统应根据任务结构、协同任务比例、截止时间要求、能耗代价和稳定性需求进行综合决策。

6.2 研究亮点

1. 本文将启发式算法与隐枚举法结合，形成了热启动后的隐枚举法，在保证求解质量的同时提高了计算效率。对于多机械臂搬运调度问题，如果直接采用完整枚举，需要遍历大量任务顺序、机械臂分配方式和双臂协同组合，随着任务数量增加，搜索空间会迅速膨胀。融合剪枝的隐枚举法虽然能够通过下界判断排除一部分无效分支，但在任务组合较复杂时，仍然需要较长的搜索时间。启发式算法则相反，它能够根据任务距离、机械臂可用时间和任务类型等规则快速构造可行解，但由于没有完整遍历搜索空间，不能保证所有算例都达到最优。

将两者结合起来，先利用启发式算法快速得到一个较优的初始调度方案，再将该方案作为隐枚举搜索的初始上界。这样做的核心作用是：在隐枚举过程中，只要某个分支的时间下界或能耗下界已经不可能优于当前启发式解，就可以提前剪去，不必继续向下搜索。因此，启发式解不再只是一个近似结果，而是进一步转化为精确搜索中的剪枝依据。实验结果表明，热启动后的隐枚举法在 16 种任务组合下均能得到与 CP-SAT 求解器一致的最优结果，同时整体计算时间低于普通融合剪枝的隐枚举法和 CP-SAT 求解器。也就是说，该方法既保留了隐枚举法的最优性，又利用启发式算法提高了搜索效率，是本文主问题中综合表现较好的算法。

意义在于，本文没有只停留在“使用求解器求解”的层面，而是根据问题结构设计了自编算法，并通过标准求解器结果进行验证。启发式算法负责快速给出较好的可行上界，隐枚举法负责保证最终结果的最优性，二者结合后形成了“先快速找到较好解，再围绕较好解进行精确搜索”的求解思路。这种思路体现了运筹学中启发式方法与精确方法结合的价值，也使本文算法部分具有更强的自主性和可解释性。

2. 本文将速度倍率作为连续变量引入模型，实现了从离散调度问题到连续速度优化问题的扩展，使研究内容更加完整。主问题主要解决的是“任务由哪只机械臂执行、任务按照什么顺序执行、双臂任务由哪两只机械臂协同完成”等离散决策问题。通过主问题可以得到二臂或三臂模式下的调度方案、最大完工时间和总能耗，但该阶段默认机械臂按照给定运行参数执行任务，并没有进一步讨论运行速度变化对时间和能耗的影响。

在实际搬运系统中，速度并不是一个可以忽略的因素。速度提高通常可以缩短任务执行时间，但可能带来更高的驱动能耗；速度降低可能减少部分驱动消耗，但也会拉长作业时间，甚至无法满足截止时间要求。因此，仅仅完成离散调度还不够，还需要进一步分析“在已经确定调度结构的基础上，机械臂以多大速度运行更合适”。本文正是基于这一点，引入单臂任务速度倍率和双臂协同任务速度倍率，将离散调度结果进一步扩展为速度-能耗非线性优化问题。

这一扩展使本文的研究从任务层调度进一步延伸到运行参数优化。主问题回答的

是“如何安排任务”，非线性延伸问题回答的是“在已有安排下如何调节速度”。二者结合后，模型不仅能比较二臂和三臂在离散调度层面的完工时间和能耗，还能进一步分析不同截止时间约束下，二臂和三臂是否可以通过调速获得更优的时间-能耗表现。问题一从平均意义上研究速度-能耗优化，问题二进一步考虑不同 seed 下任务位置变化，研究统一速度设置下的鲁棒性。这样，本文不仅分析了平均表现，也分析了最坏情形下的表现，使二臂/三臂模式选择更加充分和有效。

意义在于没有把多机械臂调度停留在单一的整数规划问题上，而是把离散优化与连续优化结合起来。离散调度解决资源分配和顺序安排，连续速度优化解决运行参数选择，鲁棒分析进一步考虑随机任务分布带来的影响。通过这种分层扩展，本文能够更全面地刻画多机械臂搬运系统中时间、能耗和稳定性之间的关系，也更符合实际工程系统中“先安排任务，再优化运行参数”的决策过程。

6.3 心得体会

通过本次大作业，我首先从代码实现层面加深了对课内知识的理解。课堂上学习整数规划、目标规划、隐枚举法、启发式算法、非线性规划和 KKT 条件时，更多是从定义、公式和例题角度理解方法；而在本项目中，这些内容都需要落实到具体变量、约束、目标函数和程序逻辑中。例如，目标规划不只是写出多个目标，而是要在程序中体现目标的优先级；隐枚举法也不是简单地枚举所有方案，而是要设计合理的下界和剪枝规则；KKT 条件也不只是理论公式，而是可以用于检查非线性优化结果是否满足一阶最优性。通过把课内方法转化为可运行的程序，我对这些方法的理解更加具体。

其次，本次作业让我认识到，运筹学建模必须从实际问题结构出发。最初看多机械臂搬运问题时，很容易把它理解为普通任务分配问题，认为只要决定每个任务由哪只机械臂执行即可。但在实际建模过程中可以发现，任务顺序、机械臂当前位置、双臂协同、资源互斥、中心安全区占用和目标优先级都会影响最终结果。因此，一个实际问题往往不能直接套用单一标准模型，而是需要先分析问题中的关键矛盾，再选择合适的数学工具进行抽象。这个过程使我更加理解了“建模”本身的重要性。

最后，本次作业也让我体会到，求解结果必须经过对比和验证。一个算法能够输出结果，并不代表结果一定可靠。对于主问题，需要将自编算法与 CP-SAT 求解器进行对照，判断其是否达到标准求解器的结果；对于非线性延伸问题，需要用 KKT 条件检查可行性、互补松弛和平稳性，判断数值解是否具有有一阶最优性。通过这种验证过程，才能说明模型和算法不是只停留在形式上，而是具有一定可信度。

从后续扩展角度看，本文仍有进一步改进空间。当前模型主要从任务层调度角度进行抽象，还没有深入考虑真实机械臂的避碰轨迹、加减速过程和动力学约束；任务集合也是静态给定的，没有考虑动态任务到达或临时任务插入；鲁棒问题目前采用逐次线性规划得到数值近似解，后续还可以引入更稳定的非线性求解器或更严格的全局优化方法进行验证。如果进一步扩展，可以使模型更加接近真实工业搬运场景。

7 思考

7.1 主问题是否可以理解为一部分动态规划？

本文主问题可以理解为具有一定动态规划思想，但不能严格称为标准动态规划问题。

从相似性来看，主问题确实存在“分阶段决策”的特征。调度过程可以看作逐步选择任务的过程：在每一步中，需要根据当前系统状态选择一个尚未完成的任务，并决定由哪只机械臂或哪两只机械臂执行。任务执行完成后，系统状态发生变化，例如已完成任务集合增加，机械臂当前位置改变，机械臂可用时间更新，中心安全区释放时间也可能发生变化。后续任务的安排又会受到这些状态变化的影响。因此，从“状态—决策—状态转移”的角度看，本文主问题与动态规划具有一定共同点。

如果将其写成动态规划思想下的状态，状态至少需要包含

$\Omega = (\text{已完成任务集合}, \text{各机械臂当前位置}, \text{各机械臂可用时间}, \text{中心安全区状态}, \text{累计能耗}, \text{负载情况})$

每一步决策则可以表示为：从未完成任务中选择一个任务，并选择对应的执行模式。对于单臂任务，需要选择一只机械臂；对于双臂协同任务，需要选择两只机械臂。决策完成后，系统进入新的状态。从这个意义上说，本文主问题确实包含动态规划的阶段决策思想。

但是，本文主问题并不适合直接采用标准动态规划求解。原因在于状态维度过高，状态数量会随着任务数量和机械臂数量迅速增加。仅“已完成任务集合”这一项就会带来指数级状态数量；再加上机械臂位置、可用时间、中心安全区占用、能耗和负载等连续或半连续信息，状态空间会变得非常庞大。如果按照标准动态规划建立状态表并逐层递推，计算量会难以接受。

此外，标准动态规划通常要求状态转移关系相对明确，并且每个状态的价值函数能够被有效存储和复用。而本文主问题中，任务执行时间和能耗不仅取决于当前任务，还取决于前序任务位置、机械臂组合、双臂同步等待和中心安全区冲突。因此，不同路径到达的状态即使已完成任务集合相同，也可能由于机械臂位置和时间不同而具有完全不同的后续代价。这使得状态合并和价值函数递推都比较困难。

因此，本文主问题更准确的理解是：它具有动态规划的阶段决策思想，但实际求解上更接近组合调度中的分支搜索和隐枚举方法。本文采用的隐枚举法也是逐步构造调度方案，但它不是建立完整动态规划状态表，而是在搜索树中结合下界和剪枝规则排除不可能成为最优解的分支。因此，主问题不能严格称为动态规划问题，但可以说它吸收了动态规划中“逐阶段决策、状态随决策变化”的思想。

7.2 匈牙利算法为何不能使用？

匈牙利算法是求解标准指派问题的经典方法，尤其适用于一类特殊的 0-1 整数规划问题。在标准指派问题中，通常有若干个执行者和若干个任务，每个执行者分配一个任务，每个任务也只由一个执行者完成，并且执行者完成任务的成本在求解之前已经给

定。其典型模型为

$$\min \sum_i \sum_j c_{ij} x_{ij},$$

其中 c_{ij} 表示执行者 i 完成任务 j 的固定成本, x_{ij} 表示是否将任务 j 分配给执行者 i 。在这种情况下, 问题的核心是静态的一对一匹配, 匈牙利算法能够高效求解。

本文主问题虽然也包含 0-1 决策变量, 但它并不是标准指派问题, 因此不能直接使用匈牙利算法。首先, 本文中的任务并不都是一对一执行。Type1 和 Type2 可以由单只机械臂完成, 但 Type3 和 Type4 需要两只机械臂协同完成。也就是说, 一个任务可能需要两个执行者同时参与, 这已经突破了标准指派问题中“一个任务对应一个执行者”的基本假设。

其次, 本文中的成本不是固定成本矩阵。某只机械臂执行某个任务所需的时间和能耗, 不仅取决于任务本身, 还取决于该机械臂上一项任务的位置、当前可用时间、是否需要等待另一只机械臂协同, 以及中心安全区是否被占用。因此, 成本会随着任务顺序和系统状态变化, 而不是求解前就固定好的 c_{ij} 。没有固定成本矩阵, 匈牙利算法就失去了直接适用的前提。

再次, 匈牙利算法只能解决“谁分配给谁”的问题, 不能解决“任务按什么顺序执行”的问题。而本文主问题中, 任务顺序是影响最大完工时间和总能耗的关键因素。同样的任务分配, 如果执行顺序不同, 机械臂的空载移动距离、等待时间、中心区冲突和最终完工时间都会不同。因此, 只做一次静态分配无法描述完整调度过程。

最后, 本文还包含机械臂资源互斥、中心安全区互斥、双臂协同同步和序贯目标规划等约束。机械臂不能同时执行多个任务, 中心安全区不能被多个任务同时占用, 双臂任务需要两只机械臂协调执行, 目标函数还要按照最大完工时间、总能耗和负载均衡的优先级依次优化。这些内容都超出了匈牙利算法的适用范围。

因此, 匈牙利算法虽然是求解标准 0-1 指派问题的有效方法, 但它适用于固定成本、静态、一对一匹配问题。本文主问题同时包含任务分配、任务排序、双臂协同、资源互斥、中心安全区冲突和多目标优先级, 因此不能直接使用匈牙利算法。最多只能在极度简化的局部场景中, 例如某一时刻将若干单臂任务临时分配给空闲机械臂时, 将其作为局部分配工具参考, 而不能作为完整主问题的求解方法。“

8 附件清单

附件一: 源代码

附件二: 求解数据汇总

附件三: 各方法单独运行调试结果

9 参考文献

[1] 学在浙大平台. 课程资料 [EB/OL]. <https://courses.zju.edu.cn/course/>.

[2] 胡运权, 郭耀煌. 运筹学教程 [M]. 北京: 清华大学出版社.

[3] Google OR-Tools. CP-SAT Solver[EB/OL]. https://developers.google.com/optimization/cp/cp_solver.

- [4] Google OR-Tools. Python Reference: ortools.sat.python.cp_model[EB/OL]. https://developers.google.com/optimization/reference/python/sat/python/cp_model.
- [5] Dias M B, Zlot R, Kalra N, Stentz A. Market-Based Multirobot Coordination: A Survey and Analysis[J]. Proceedings of the IEEE, 2006, 94(7): 1257–1270.
- [6] Gerkey B P, Mataric M J. Sold!: Auction Methods for Multirobot Coordination[J]. IEEE Transactions on Robotics and Automation, 2002, 18(5): 758–768.
- [7] Van den Berg J, Lin M, Manocha D. Reciprocal Velocity Obstacles for Real-Time Multi-Agent Navigation[C]. 2008 IEEE International Conference on Robotics and Automation, 2008: 1928–1935.
- [8] Van den Berg J, Guy S J, Lin M, Manocha D. Optimal Reciprocal Collision Avoidance for Multi-Agent Navigation[C]. 2011 IEEE International Conference on Robotics and Automation, 2011.
- [9] Wu N Q, Zhou M C. Modeling and Deadlock Control of Automated Guided Vehicle Systems[J]. IEEE/ASME Transactions on Mechatronics, 2004, 9(1): 50–57.
- [10] Wu N Q, Zhou M C. Shortest Routing of Bidirectional Automated Guided Vehicles Avoiding Deadlock and Blocking[J]. IEEE/ASME Transactions on Mechatronics, 2007, 12(1): 63–72.
- [11] Nishi T, Maeno R. Petri Net Decomposition Approach to Optimization of Route Planning Problems for AGV Systems[J]. IEEE Transactions on Automation Science and Engineering, 2010, 7(3): 523–537.
- [12] Nocedal J, Wright S J. *Numerical Optimization*[M]. 2nd ed. New York: Springer, 2006.

10 分工

1. 汇报：王沐灵
2. 数学建模：傅安澜 50%、修嘉卿 50%
3. 主问题 13（CP-SAT 与启发式算法）（30%）：
王沐灵 60%、傅安澜 20%、修嘉卿 20%
4. 主问题 24（融合剪枝的隐枚举法与热启动后的隐枚举法）（35%）：
傅安澜 70%、修嘉卿 20%、王沐灵 10%
5. 延伸问题（35%）：修嘉卿 60%、王沐灵 30%、傅安澜 10%